

Estructuras de Datos

Árboles Balanceados

Construya el ABB para:

□ Claves: 30, 40, 10, 50, 20, 5, 35

□ Claves: 50, 40, 35, 30, 20, 10, 5

Construya el ABB para:

▣ Claves: 30, 40, 10, 50, 20, 5, 35

▣ Claves: 50, 40, 35, 30, 20, 10, 5

¿Cuántas comparaciones se requieren para encontrar el 5?

La altura del árbol es un factor crítico en la eficiencia de la búsqueda

EFICIENCIA DE LOS ABB

□ Los ABB se utilizan

- Para almacenar datos en forma ordenada
- Y facilitar la búsqueda de información

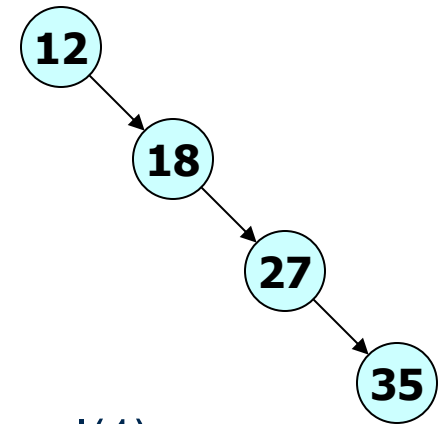
□ La estructura de un ABB

- Depende del orden en que se añaden sus nodos
- Si todos los elementos se añaden en orden creciente:
- El arbol queda alineado todo hacia la derecha

□ Y la busqueda ya no es binaria(2 posibilidades), es lineal(1)

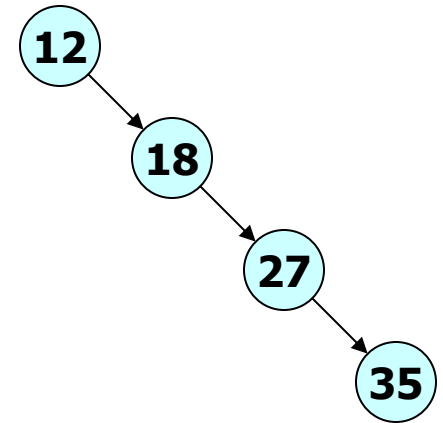
Ejemplo:

12, 18, 27 y 35

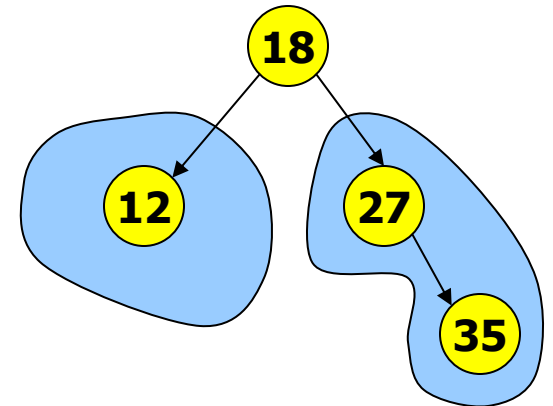


MAS EFICIENCIA

- El arbol debe guardar otra estructura
- Algo mas equilibrado,
 - Asegurar siempre una busqueda binaria
 - Que a partir de un nodo N con clave K
 - *Aprox. la mitad tengan claves mayores que K*
 - *Y aprox. la otra mitad tengan claves menores*
- Si se asegura equilibrio, se asegura eficiencia



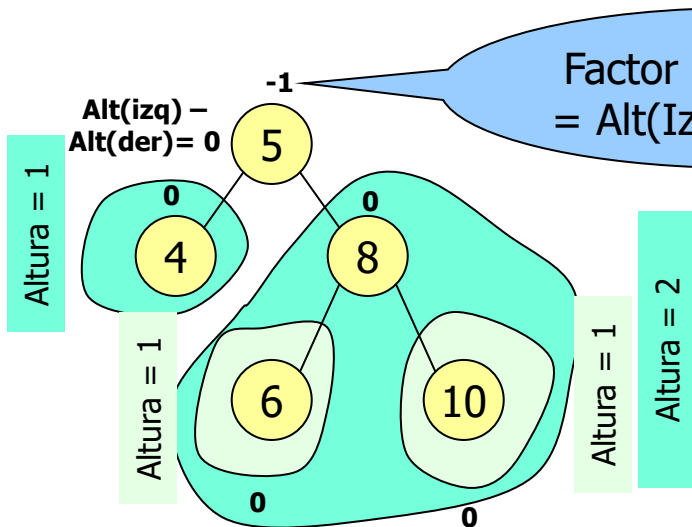
**ABB Ineficiente,
Busqueda lineal**



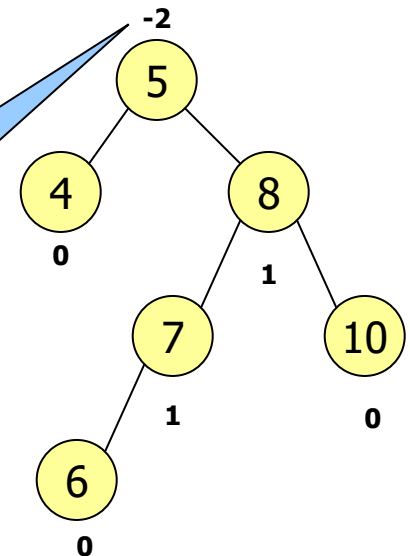
**ABB Eficiente, la busqueda
siempre es binaria**

ARBOLES EQUILIBRADOS (AVL)

- Es un Arbol Binario especial, en donde
 - Para c/Nodo **p** del arbol
 - La $Altura(p \rightarrow izq)$ y $Altura(p \rightarrow der)$
 - Nunca difieren entre si en mas de una unidad



En equilibrio, F.E. Debe ser -1, 0 o 1



ARBOL NO EQUILIBRADO

También conocidos como árboles AVL en honor a sus creadores:

Georgy Adelson-Velsky and Evgenii Landis

También conocidos como árboles AVL en honor a sus creadores:

Georgy Adelson-Velsky and Evgenii Landis



Tres claves: 30, 20, 10

Tres claves: 30, 20, 10

$3!$ = Seis posibles combinaciones

Tres claves: 30, 20, 10

30, 20, 10

30, 10, 20

10, 30, 20

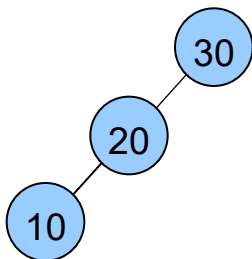
10, 20, 30

20, 10, 30

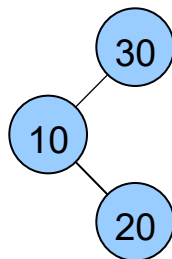
20, 30, 10

Tres claves: 30, 20, 10

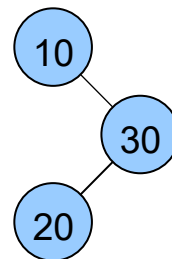
30, 20, 10



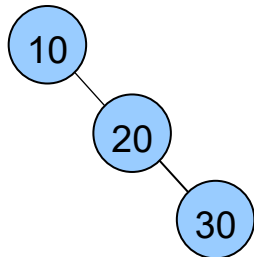
30, 10, 20



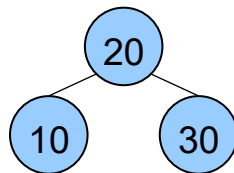
10, 30, 20



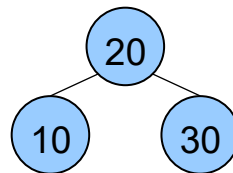
10, 20, 30



20, 10, 30

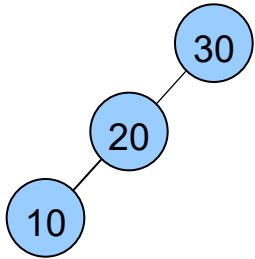


20, 30, 10

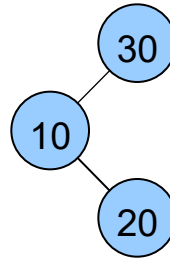


Tres claves: 30, 20, 10

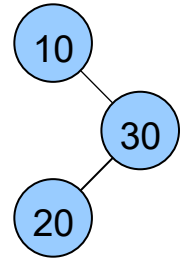
30, 20, 10



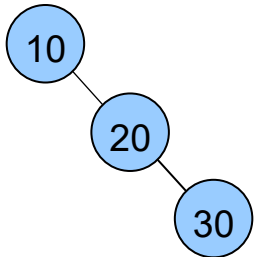
30, 10, 20



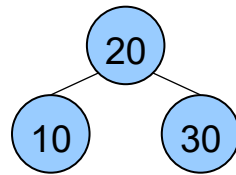
10, 30, 20



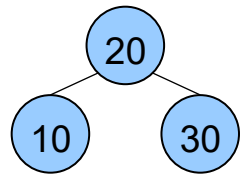
10, 20, 30



20, 10, 30

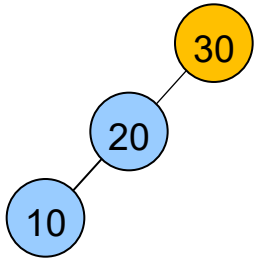


20, 30, 10

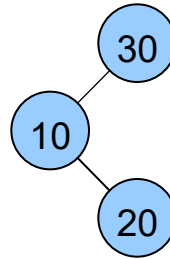


Tres claves: 30, 20, 10

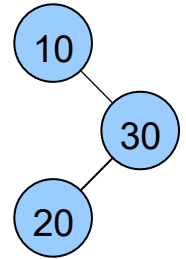
30, 20, 10



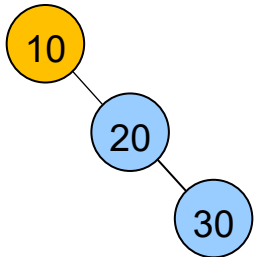
30, 10, 20



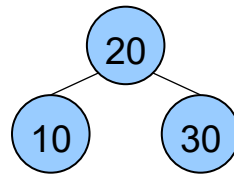
10, 30, 20



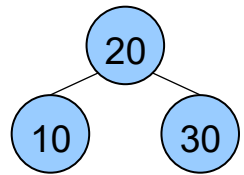
10, 20, 30



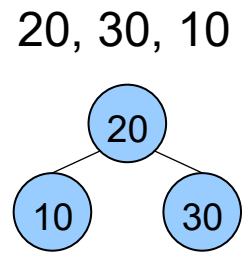
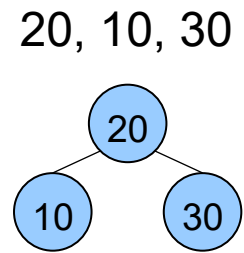
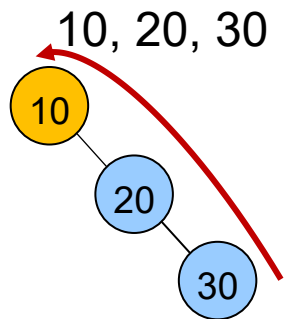
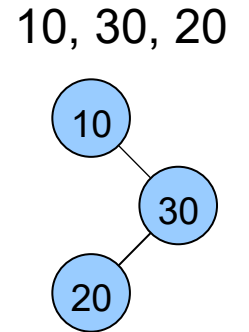
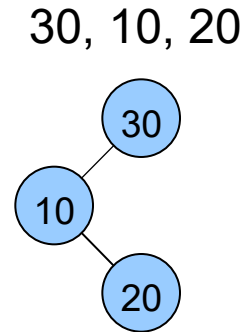
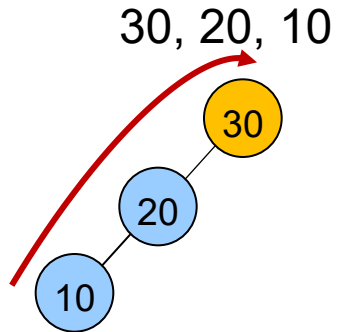
20, 10, 30



20, 30, 10

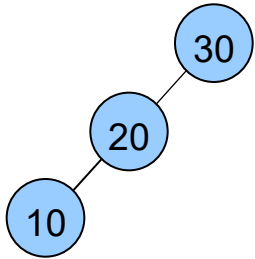


Tres claves: 30, 20, 10

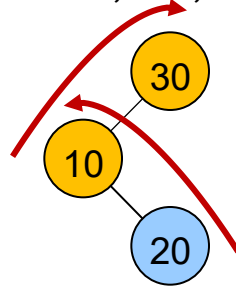


Tres claves: 30, 20, 10

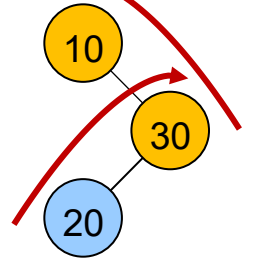
30, 20, 10



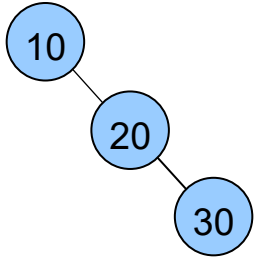
30, 10, 20



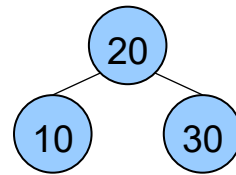
10, 30, 20



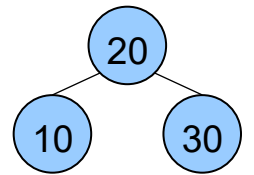
10, 20, 30



20, 10, 30

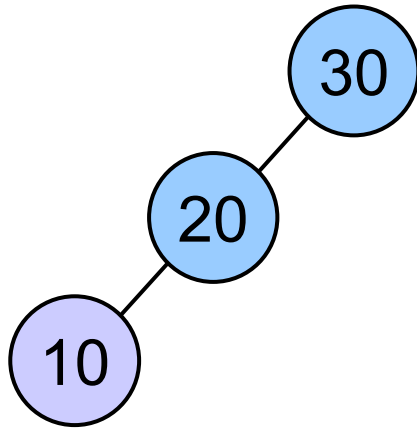


20, 30, 10

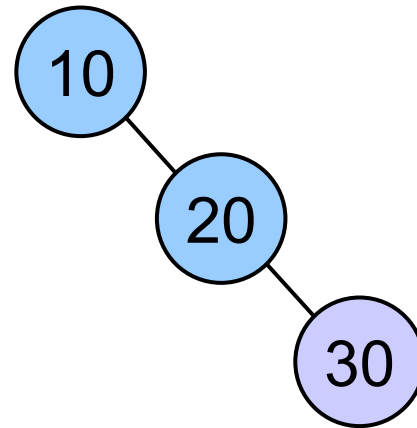


Rotación Simple

Cuando hay inserción:

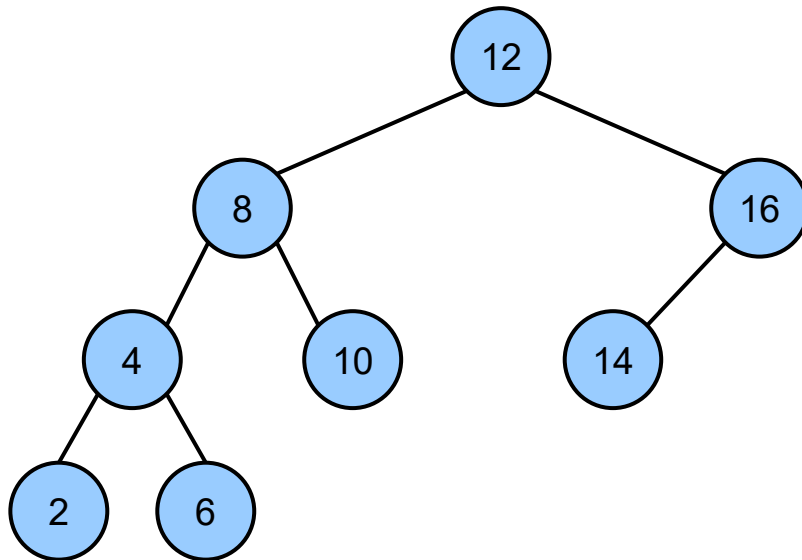


Izquierda - izquierda

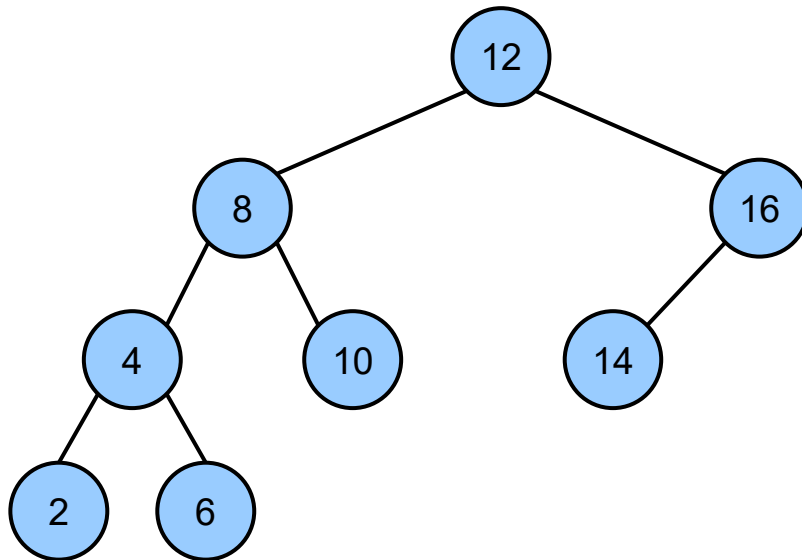


Derecha - derecha

Inserción izquierda – izquierda => Rotación (simple) hacia la derecha



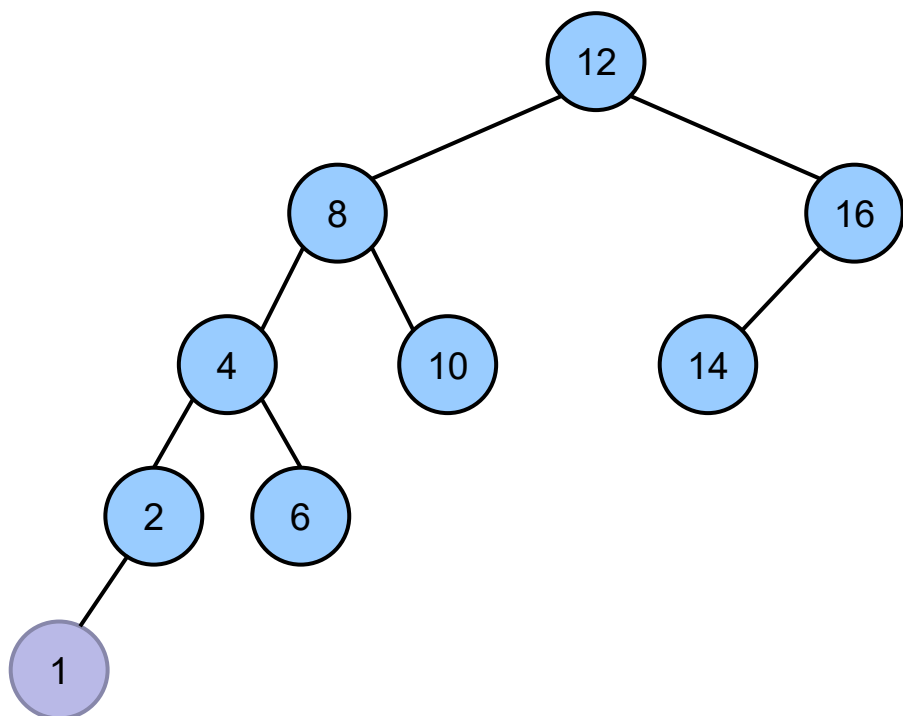
Inserción izquierda – izquierda => Rotación (simple) hacia la derecha



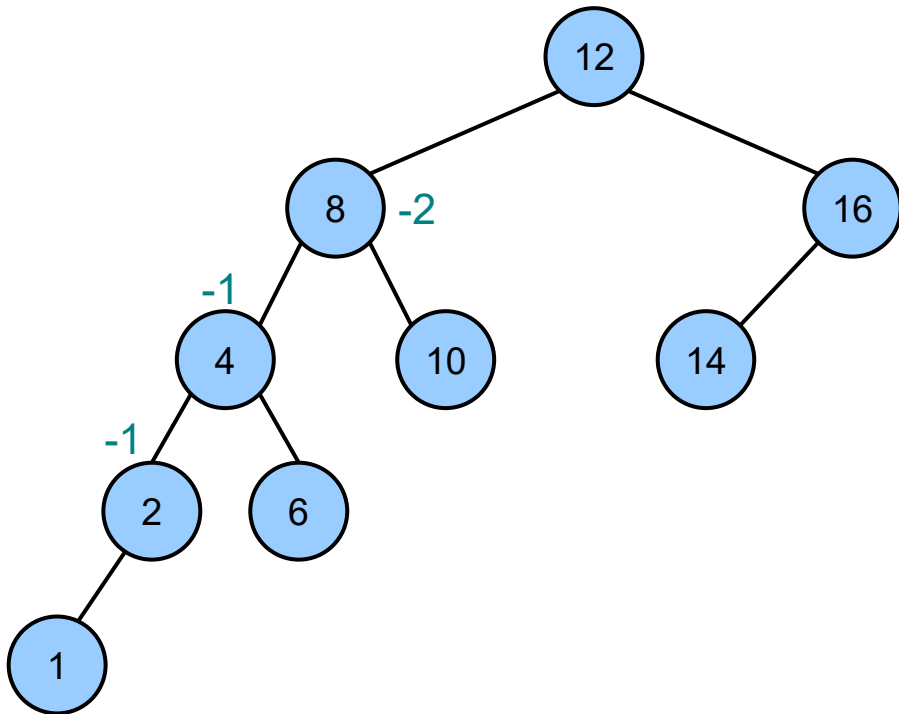
Nuevo nodo:

1

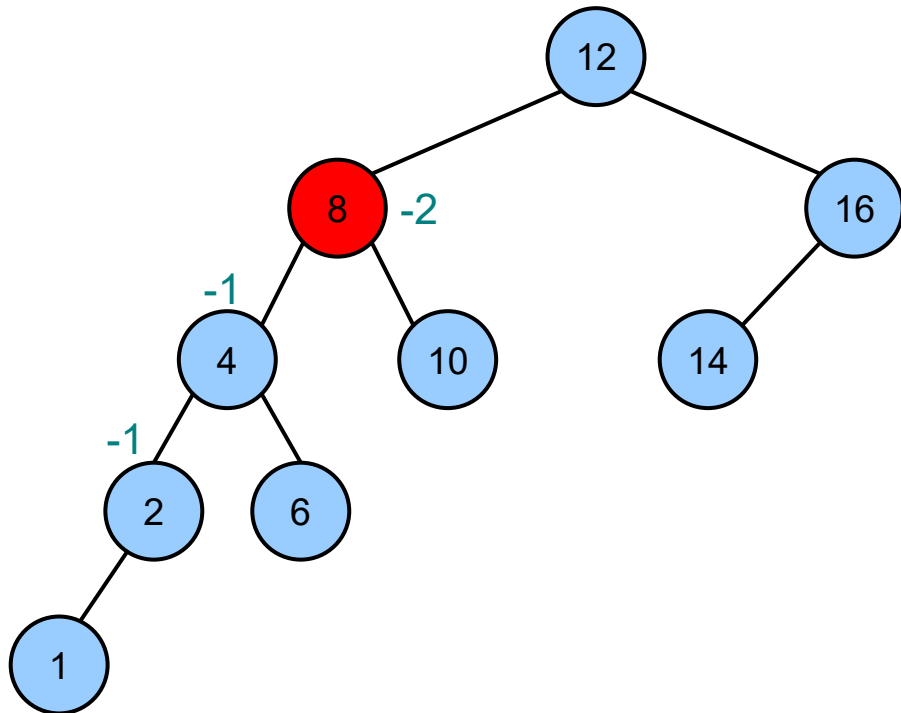
Inserción izquierda – izquierda => Rotación (simple) hacia la derecha



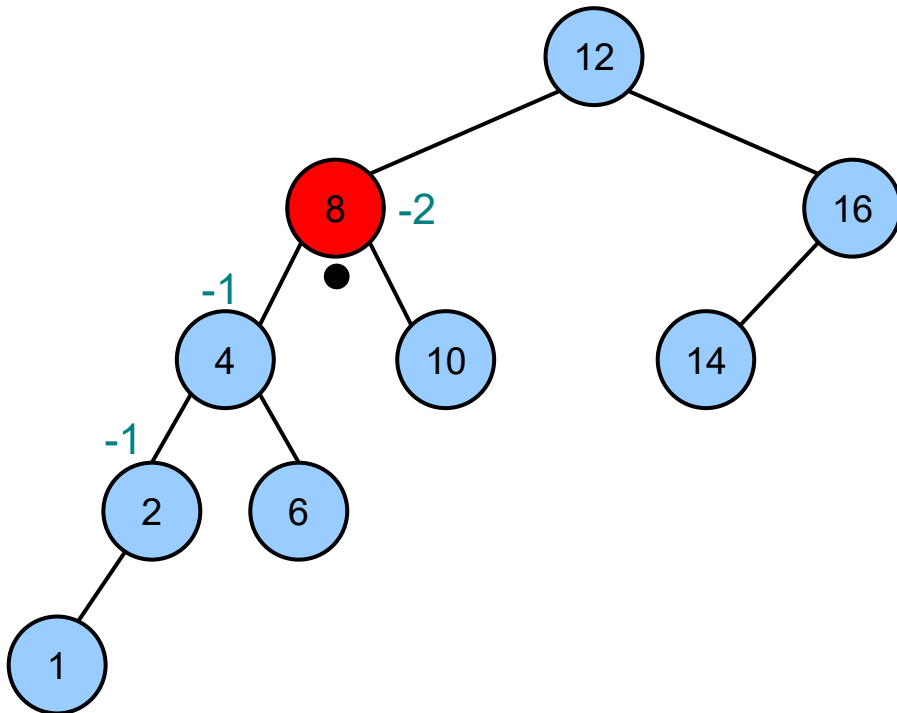
Inserción izquierda – izquierda => Rotación (simple) hacia la derecha



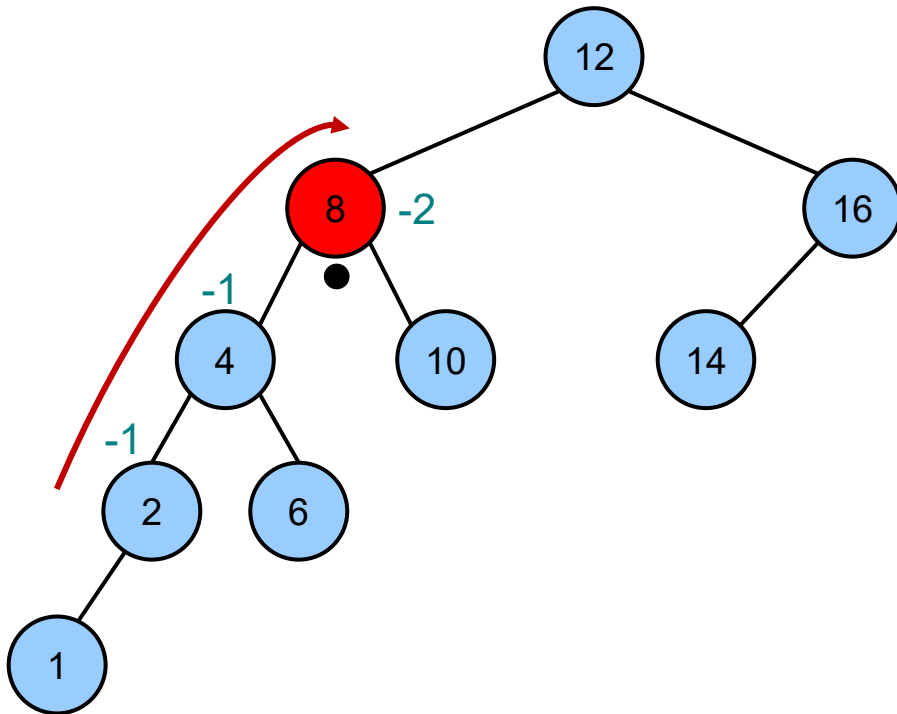
Inserción izquierda – izquierda => Rotación (simple) hacia la derecha



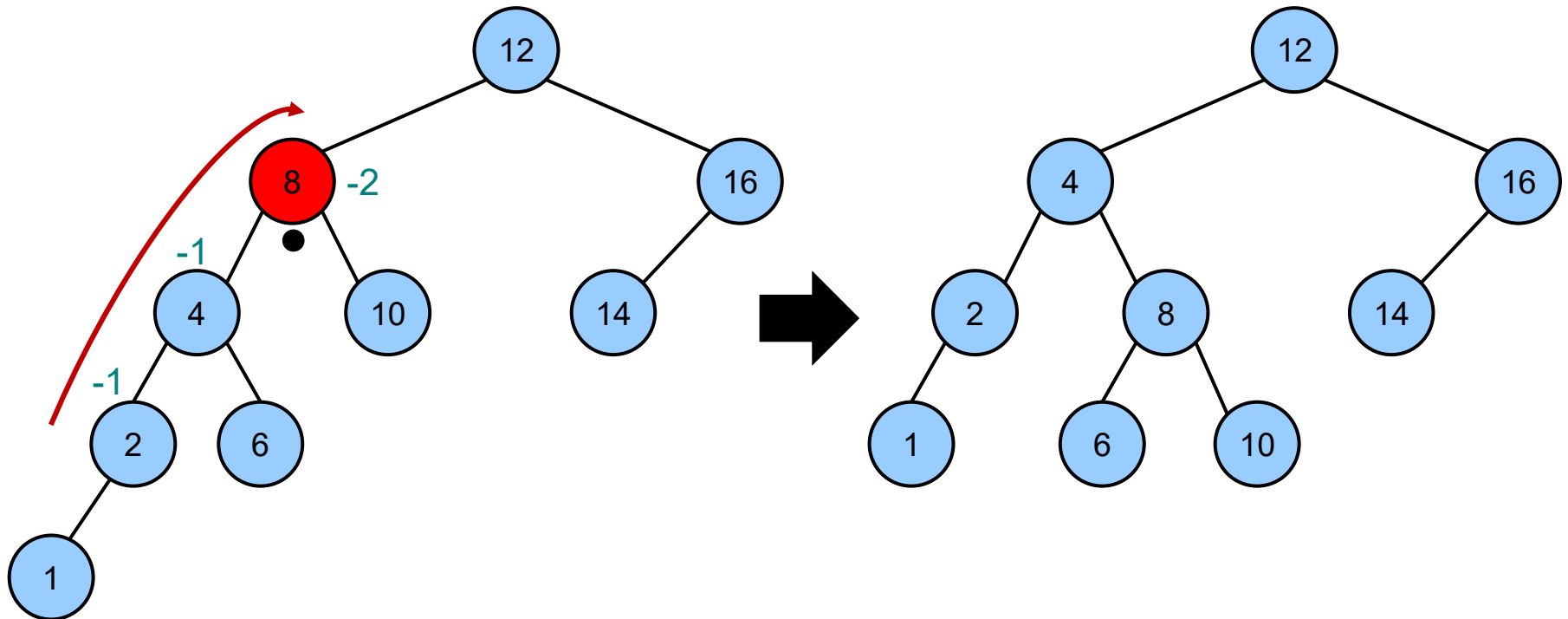
Inserción izquierda – izquierda => Rotación (simple) hacia la derecha



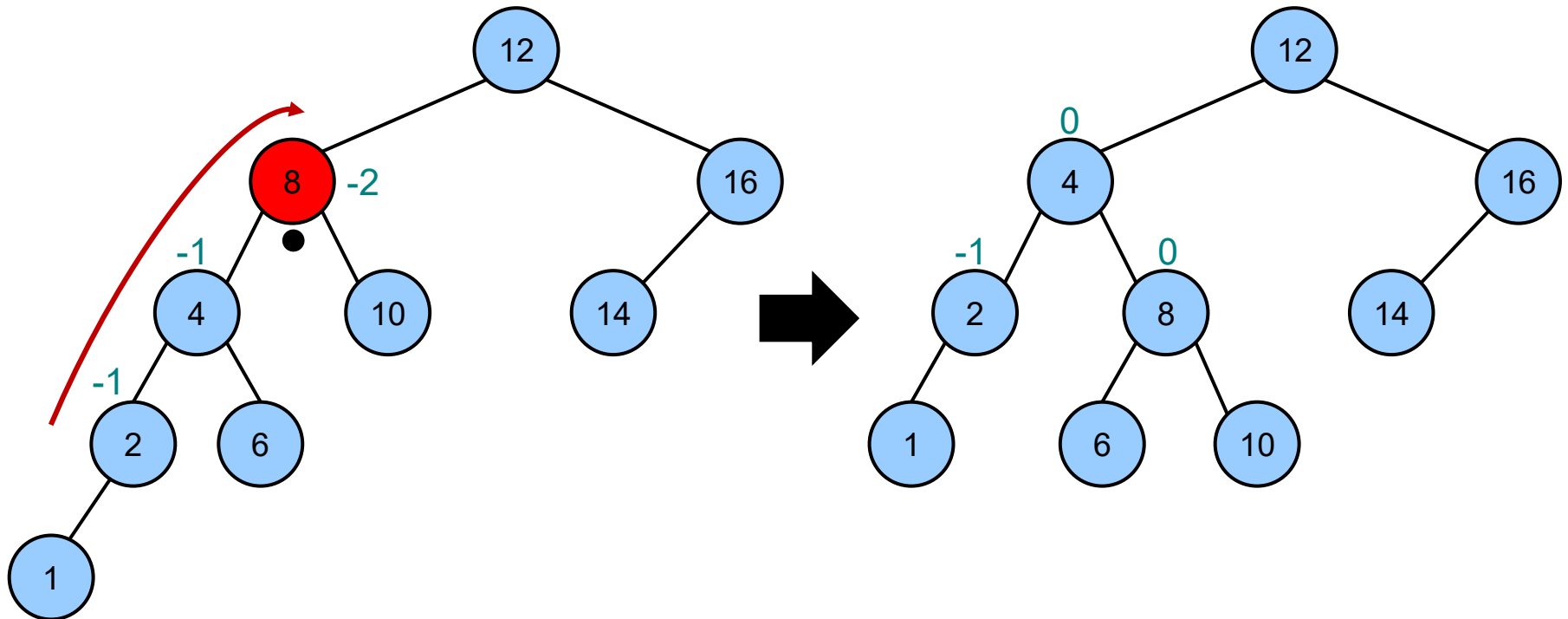
Inserción izquierda – izquierda => Rotación (simple) hacia la derecha



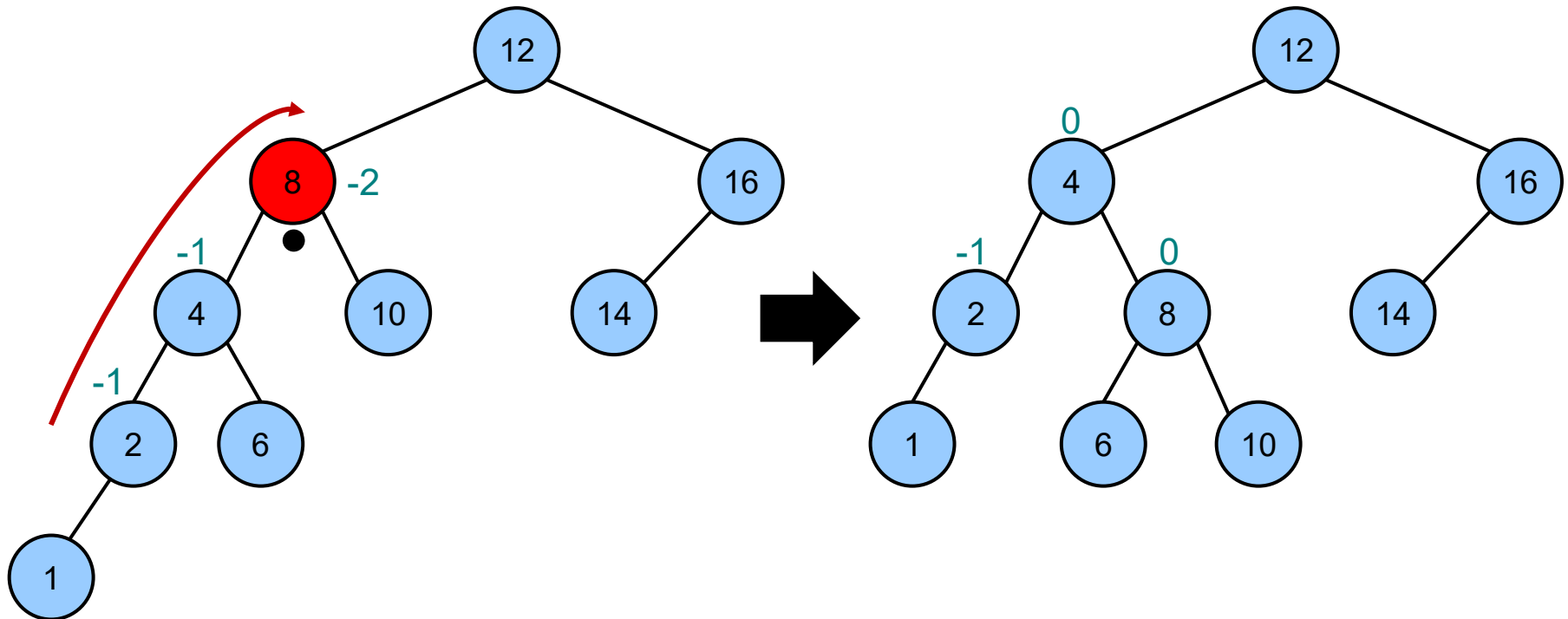
Inserción izquierda – izquierda => Rotación (simple) hacia la derecha



Inserción izquierda – izquierda => Rotación (simple) hacia la derecha



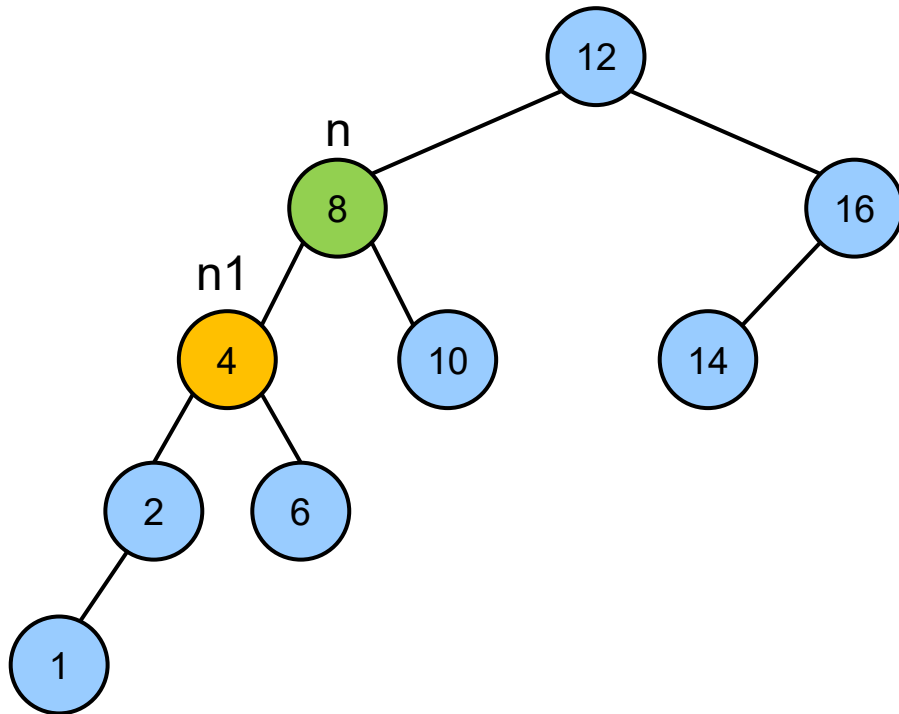
Inserción izquierda – izquierda => Rotación (simple) hacia la derecha



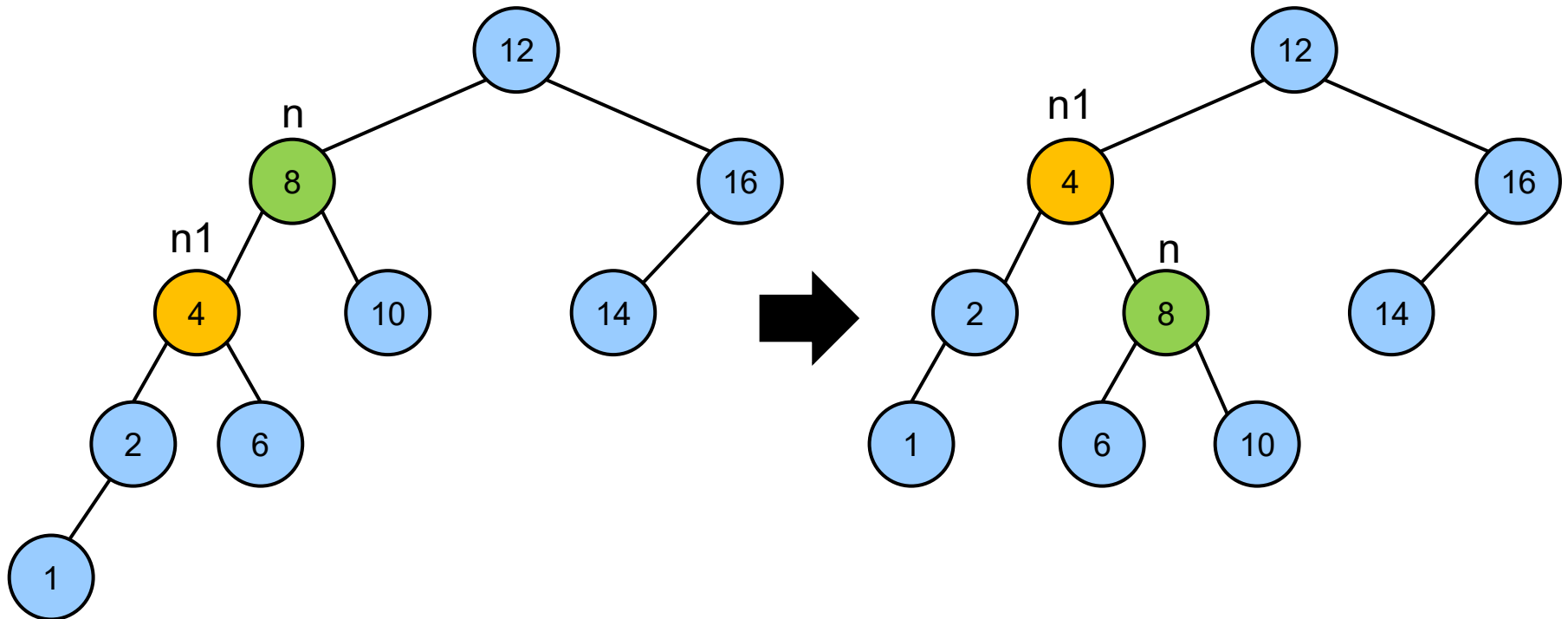
Importante:

Note que la inserción completa aquí fue: izquierda, izquierda, izquierda, izquierda
Solo se toman en cuenta las dos primeras: **izquierda-izquierda**

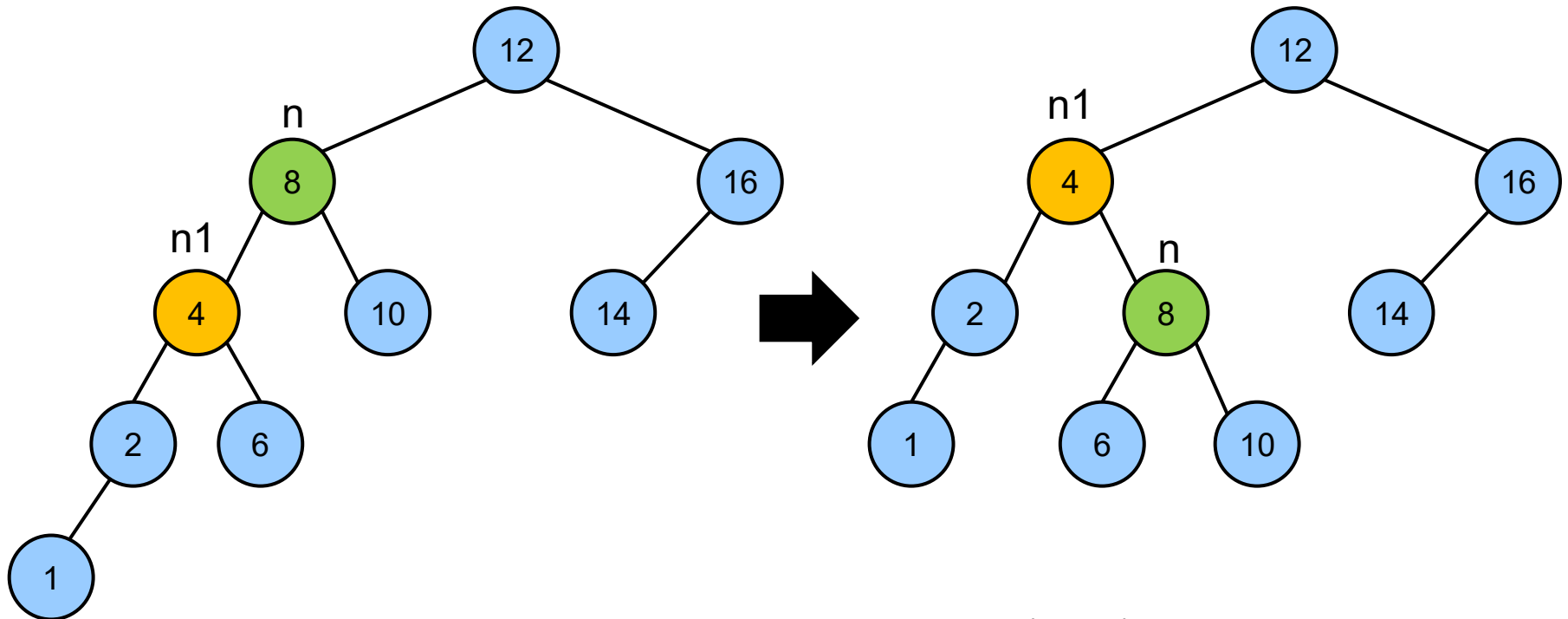
Inserción izquierda – izquierda => Rotación (simple) hacia la derecha



Inserción izquierda – izquierda => Rotación (simple) hacia la derecha



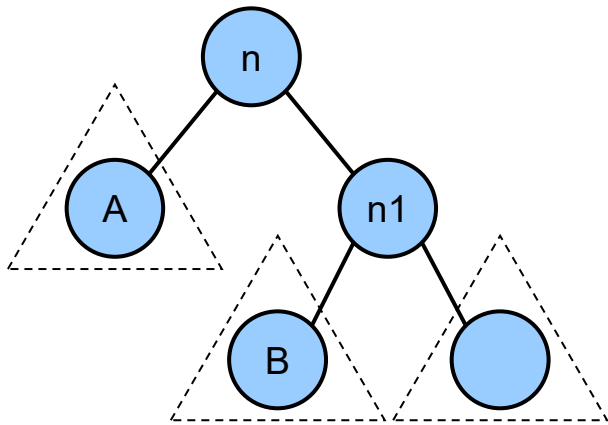
Inserción izquierda – izquierda => Rotación (simple) hacia la derecha



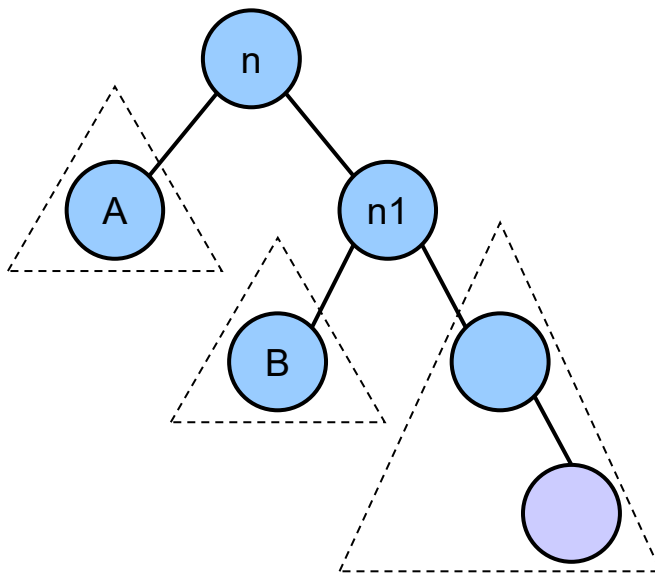
```
n.izquierdo = n1.derecho  
n1.derecho = n  
n = n1
```

La inserción derecha – derecha sigue un procedimiento similar, pero requiere una rotación izquierda para re-balancear el árbol

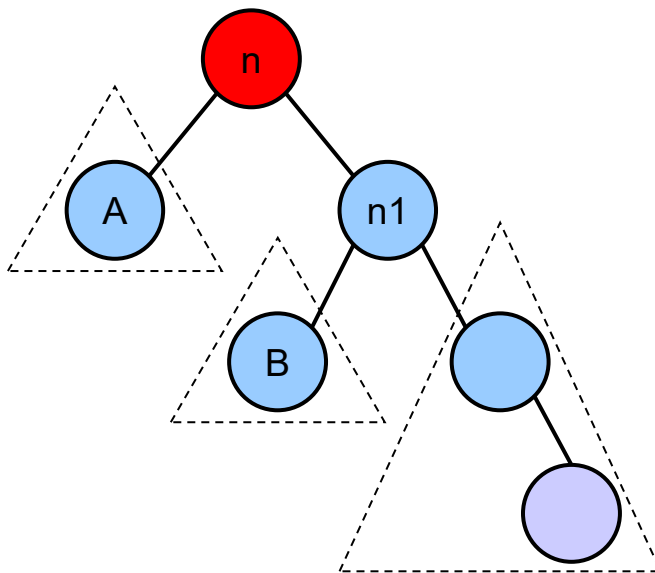
La inserción derecha – derecha sigue un procedimiento similar, pero requiere una rotación izquierda para re-balancear el árbol



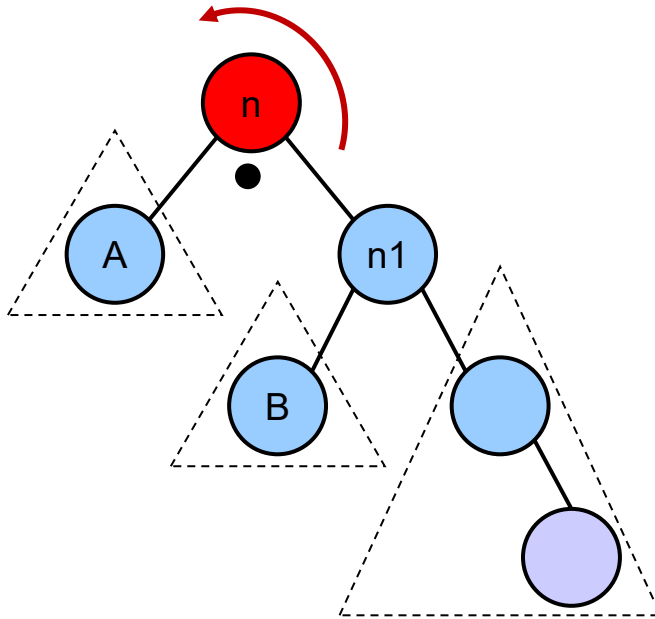
La inserción derecha – derecha sigue un procedimiento similar, pero requiere una rotación izquierda para re-balancear el árbol



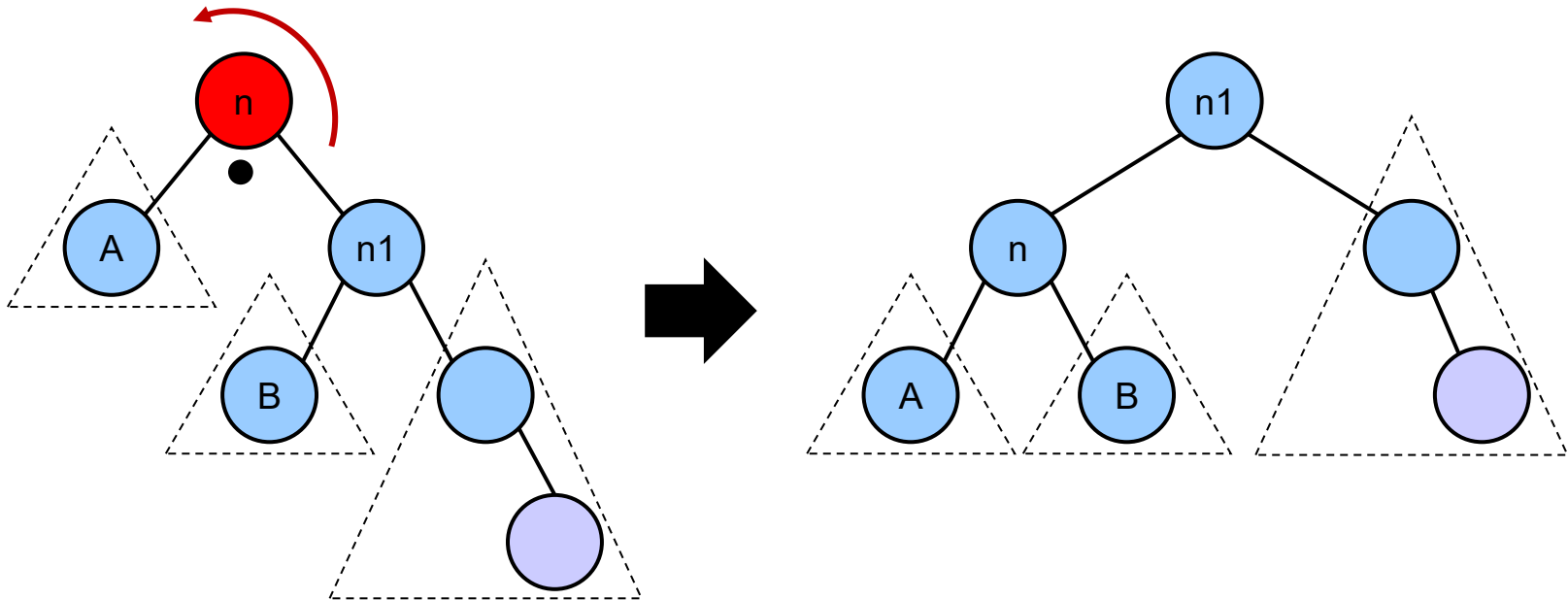
La inserción derecha – derecha sigue un procedimiento similar, pero requiere una rotación izquierda para re-balancear el árbol



La inserción derecha – derecha sigue un procedimiento similar, pero requiere una rotación izquierda para re-balancear el árbol

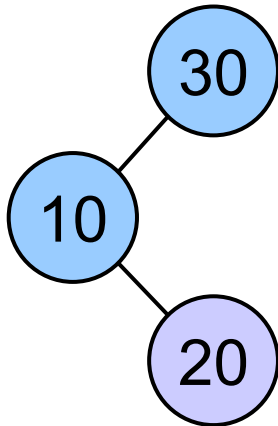


La inserción derecha – derecha sigue un procedimiento similar, pero requiere una rotación izquierda para re-balancear el árbol

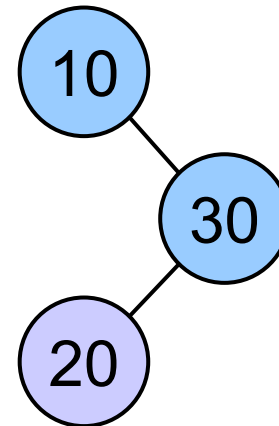


Rotación Doble

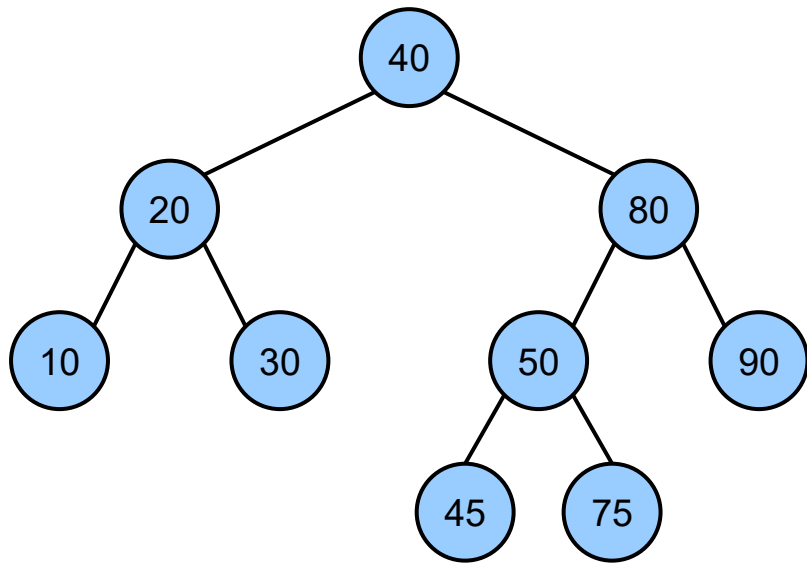
Cuando hay inserción:

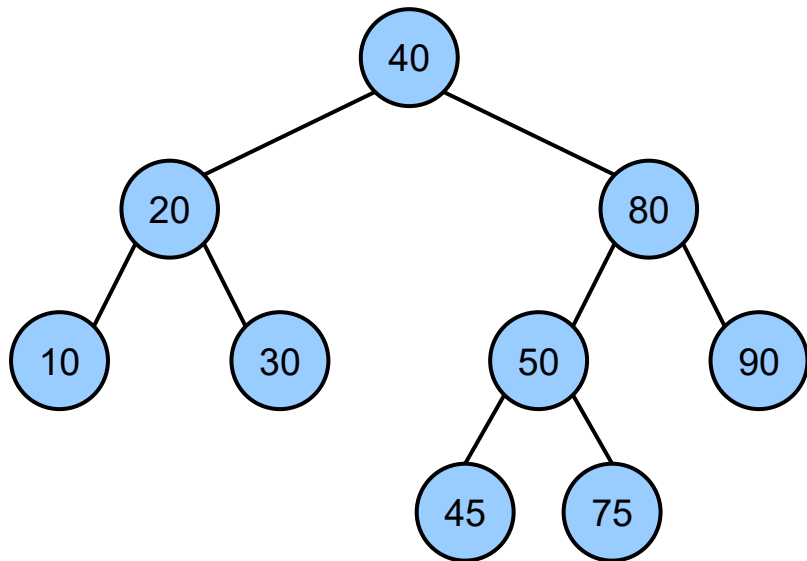


Izquierda - Derecha



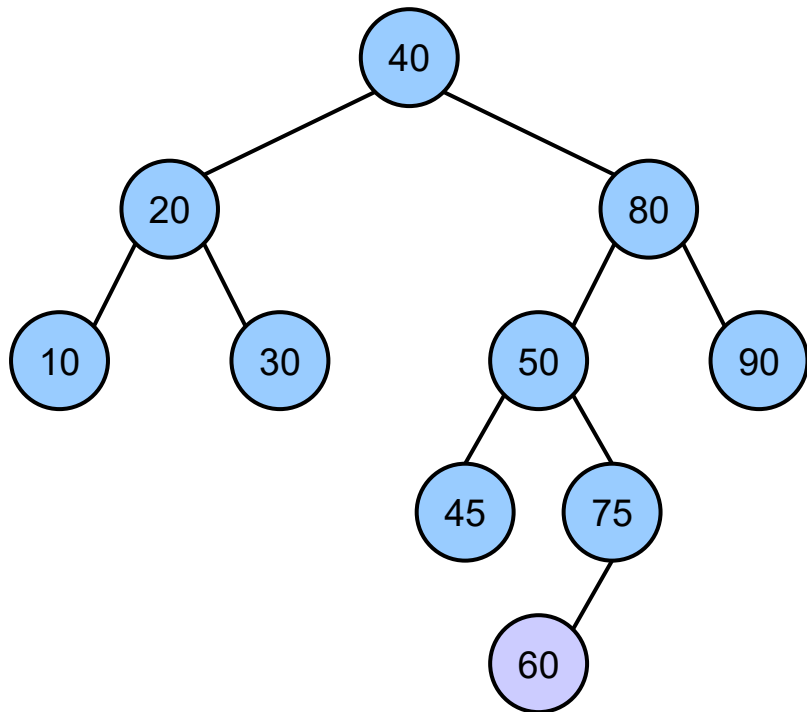
Derecha - Izquierda

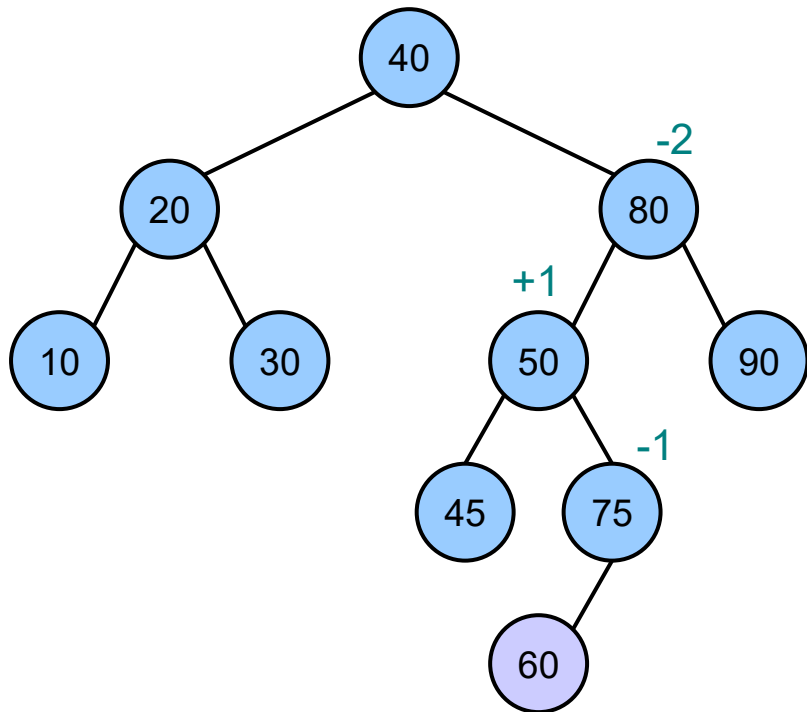


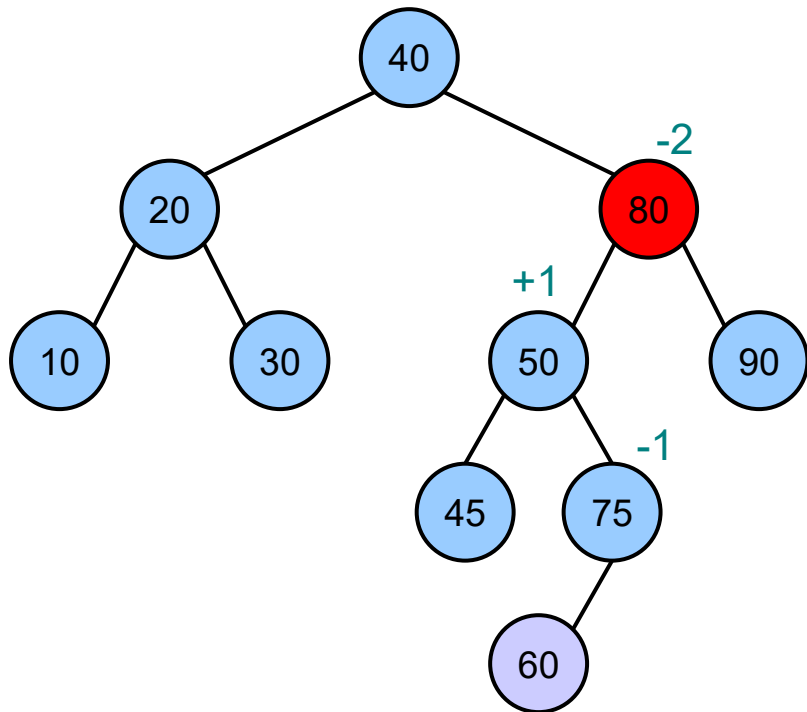


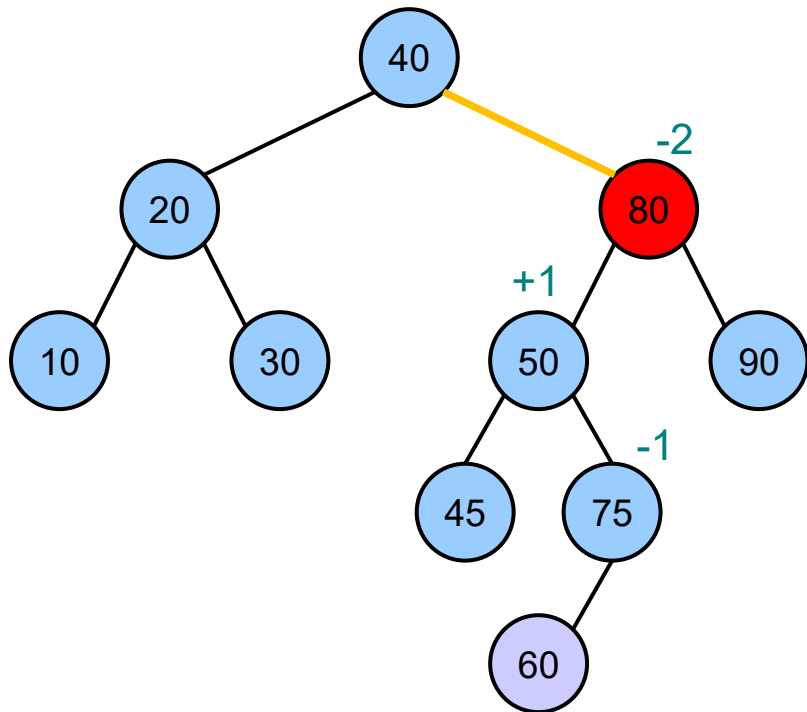
Nuevo nodo:

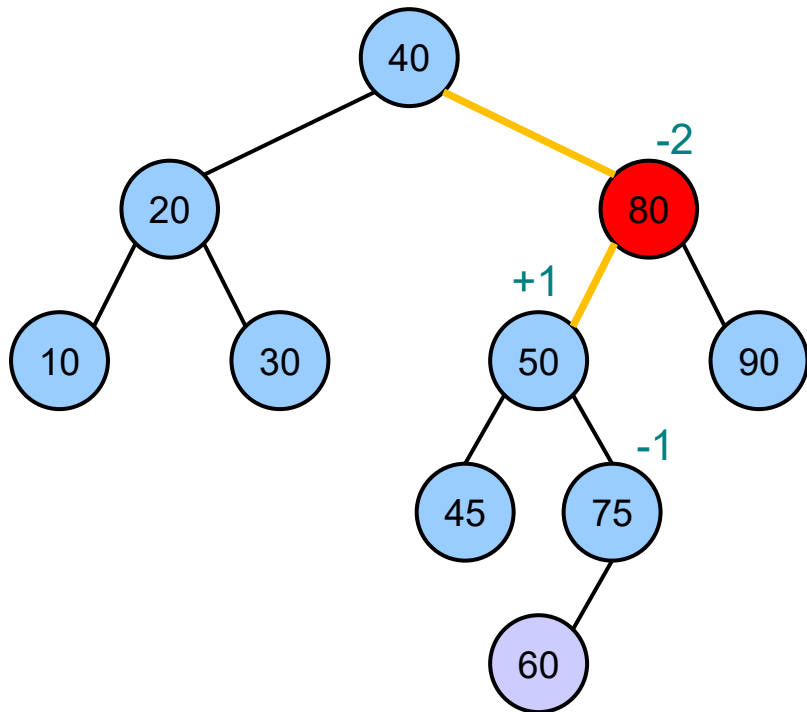
60

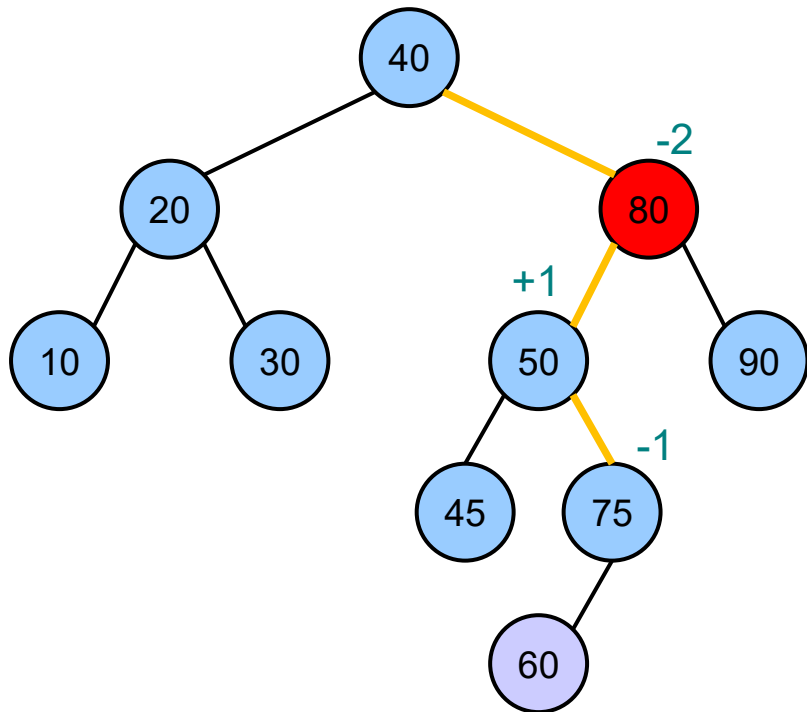


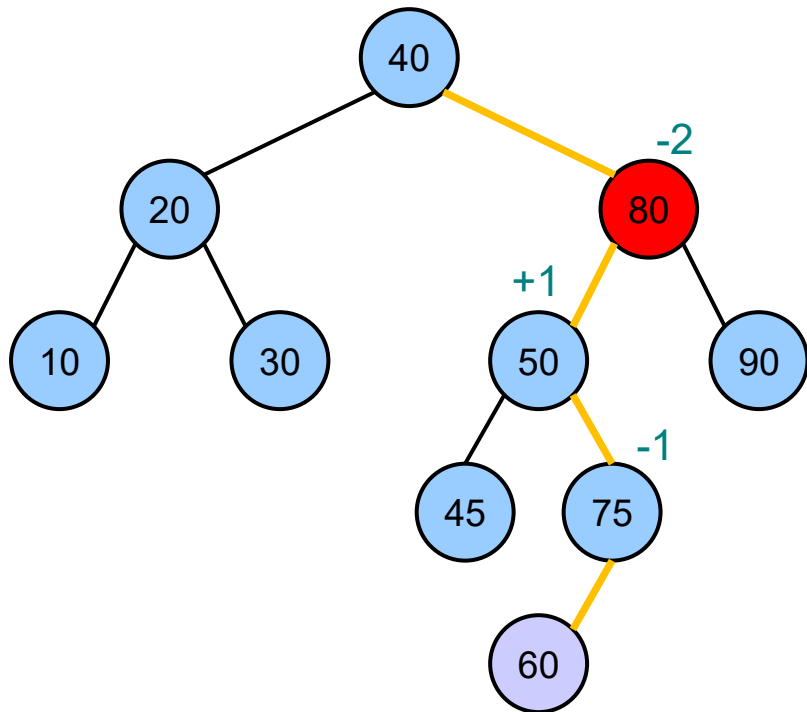


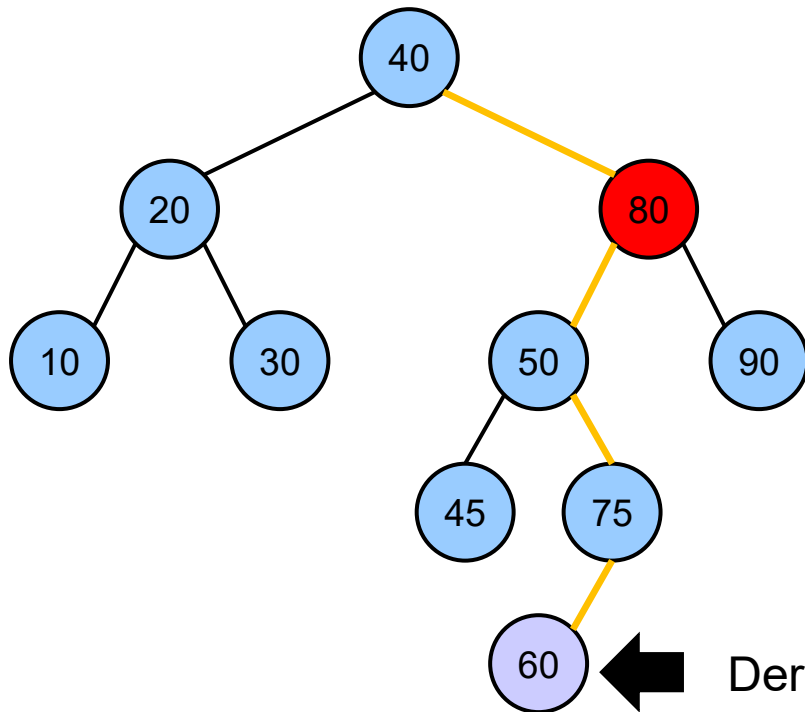




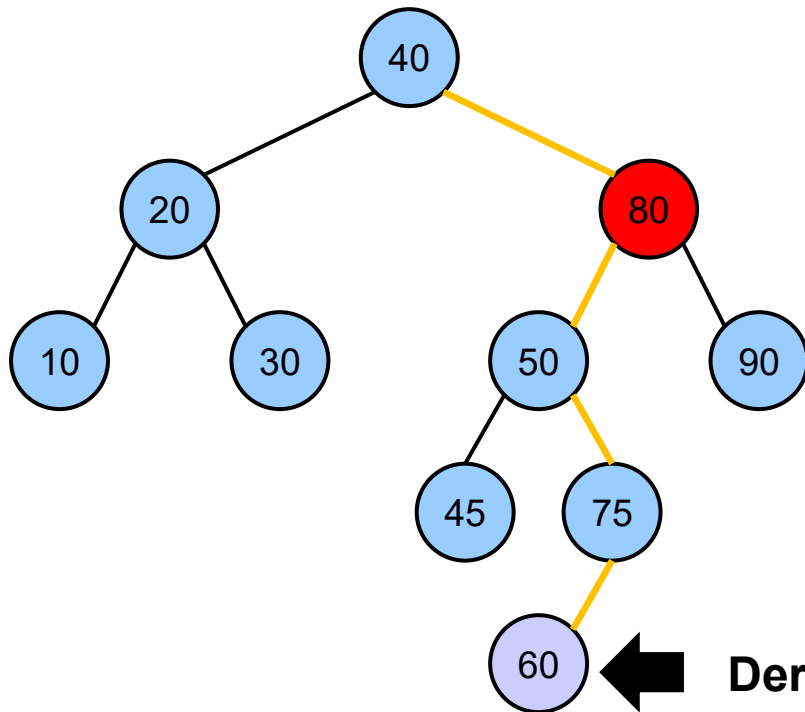




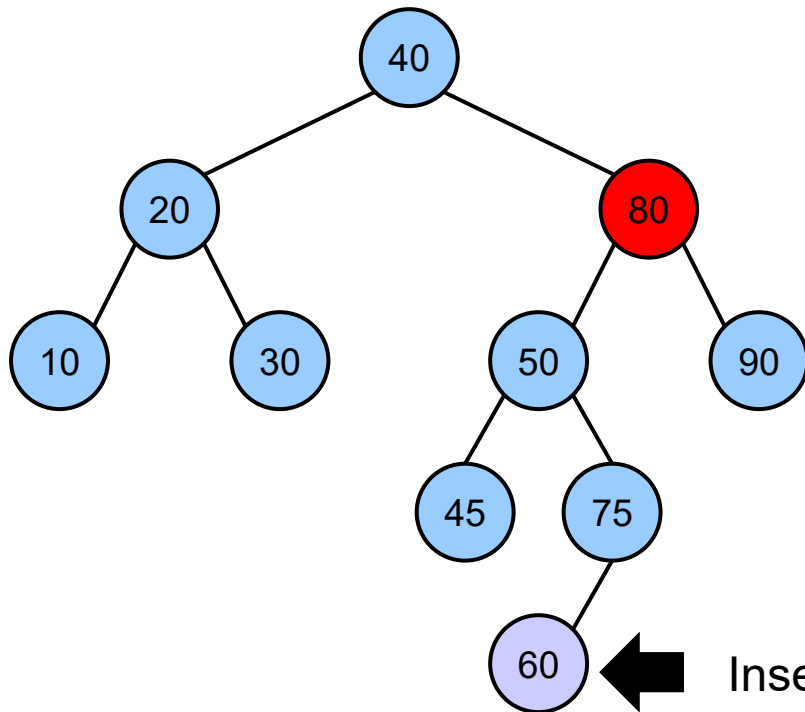




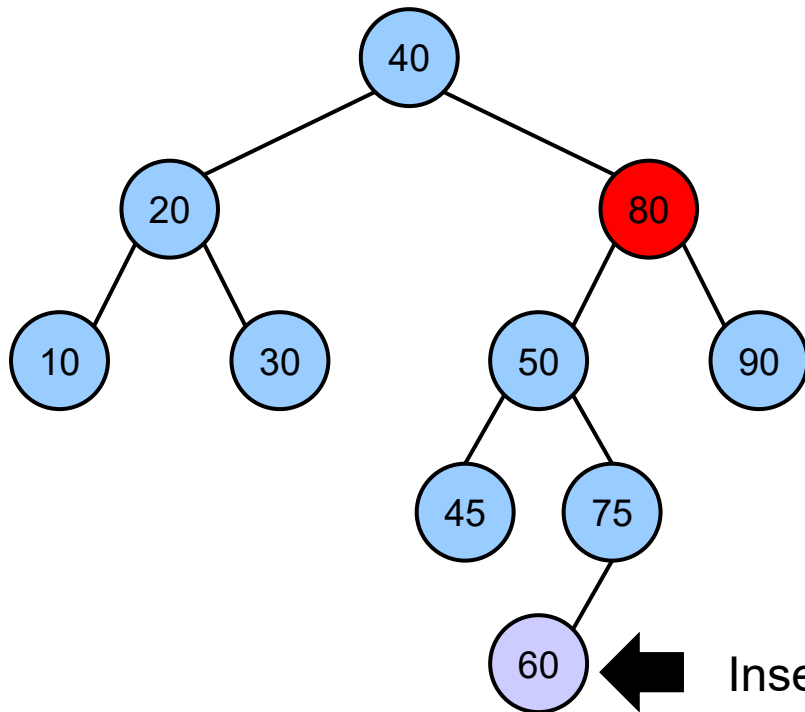
← Derecha, izquierda, derecha, izquierda



Derecha, izquierda, derecha, izquierda

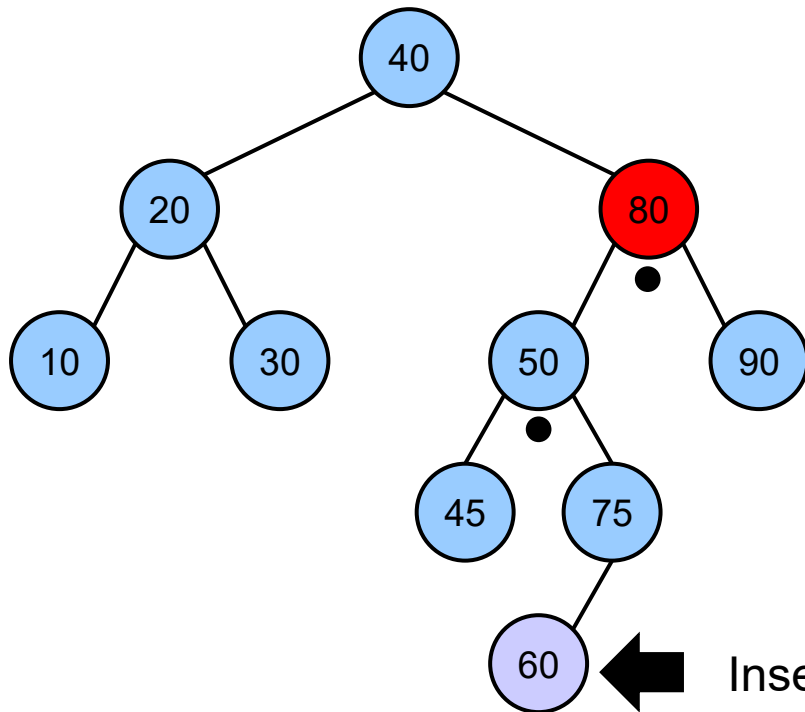


← Inserción **Derecha - Izquierda**



Insertión **Derecha - Izquierda**

Se arregla con rotación **Izquierda - Derecha**

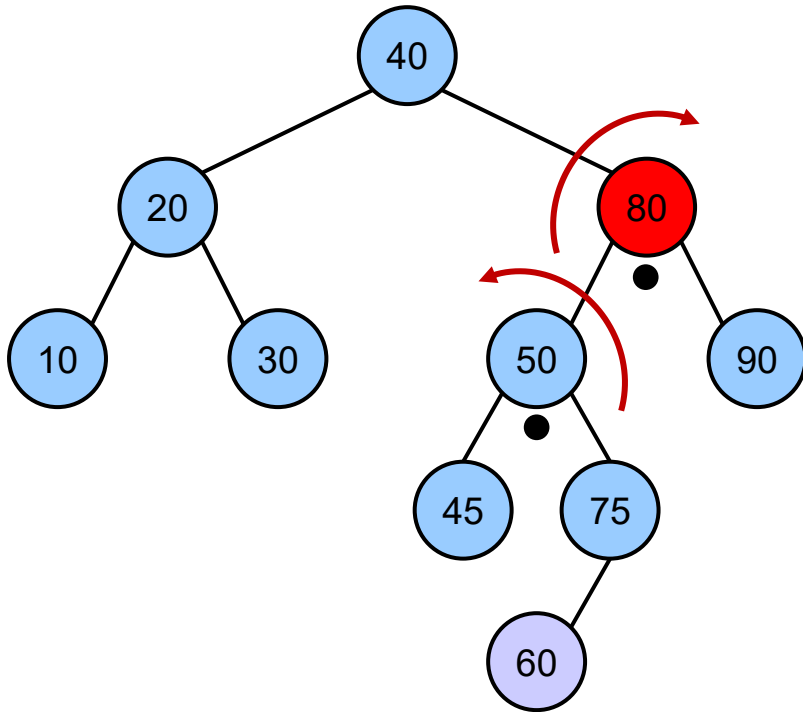


Inserción **Derecha - Izquierda**

Se arregla con rotación **Izquierda - Derecha**

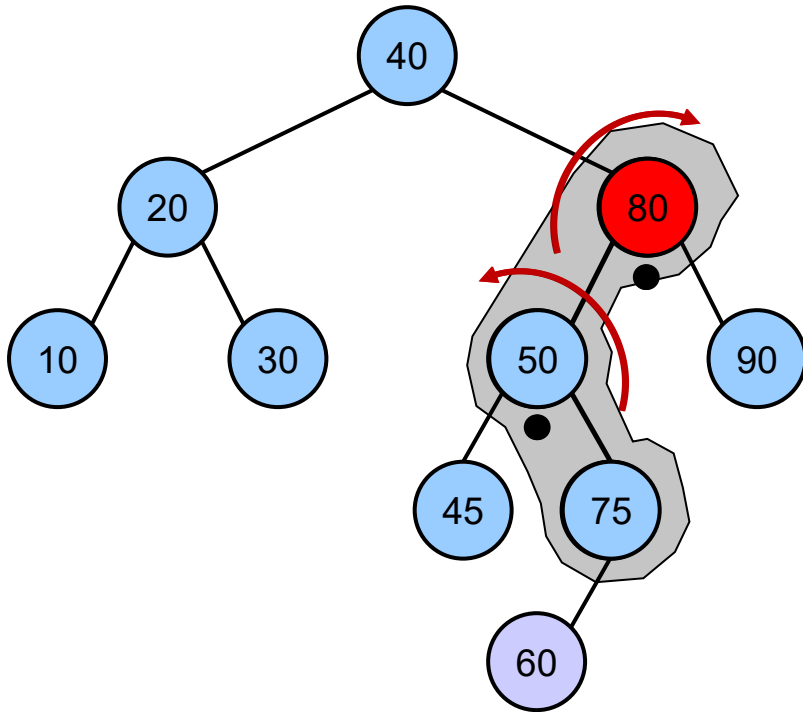
Inserción **Derecha - Izquierda**

Se arregla con rotación **Izquierda - Derecha**



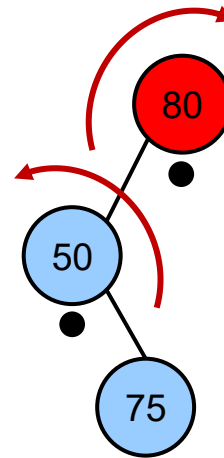
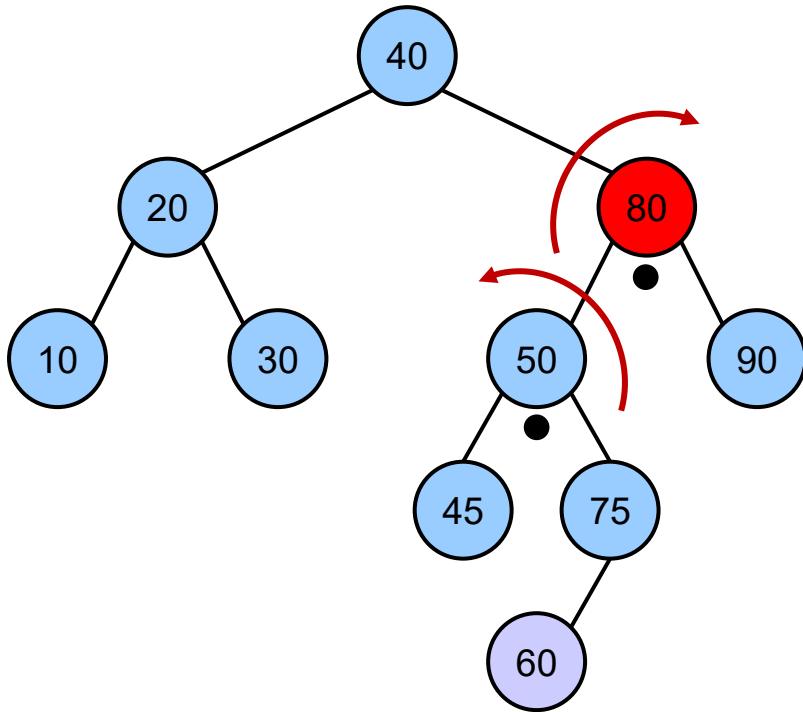
Inserción **Derecha - Izquierda**

Se arregla con rotación **Izquierda - Derecha**



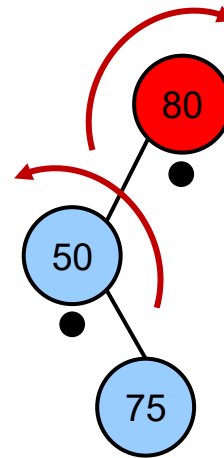
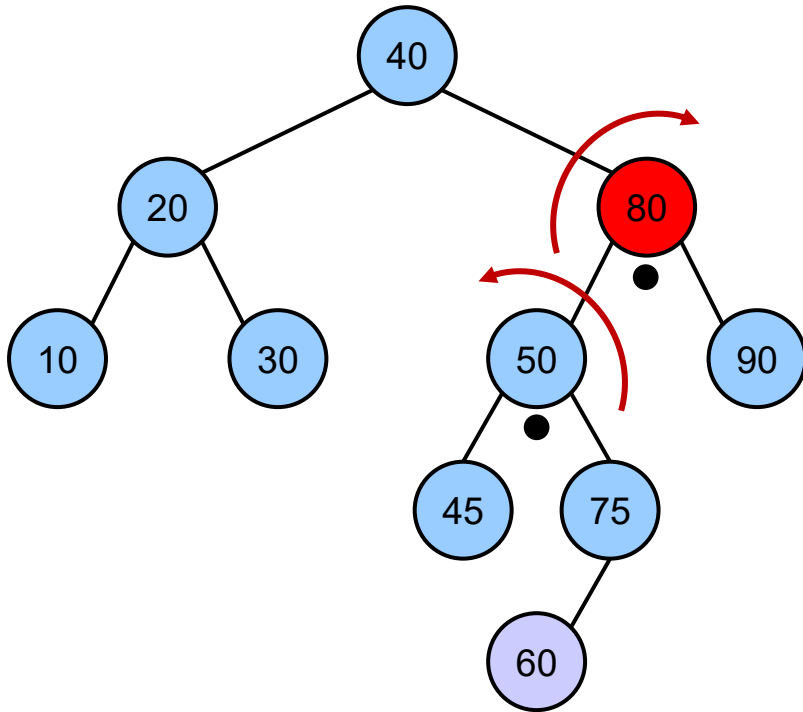
Inserción **Derecha - Izquierda**

Se arregla con rotación **Izquierda - Derecha**



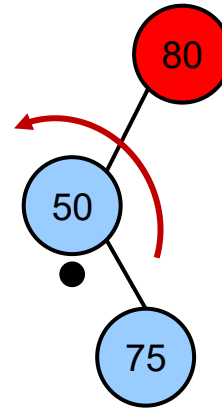
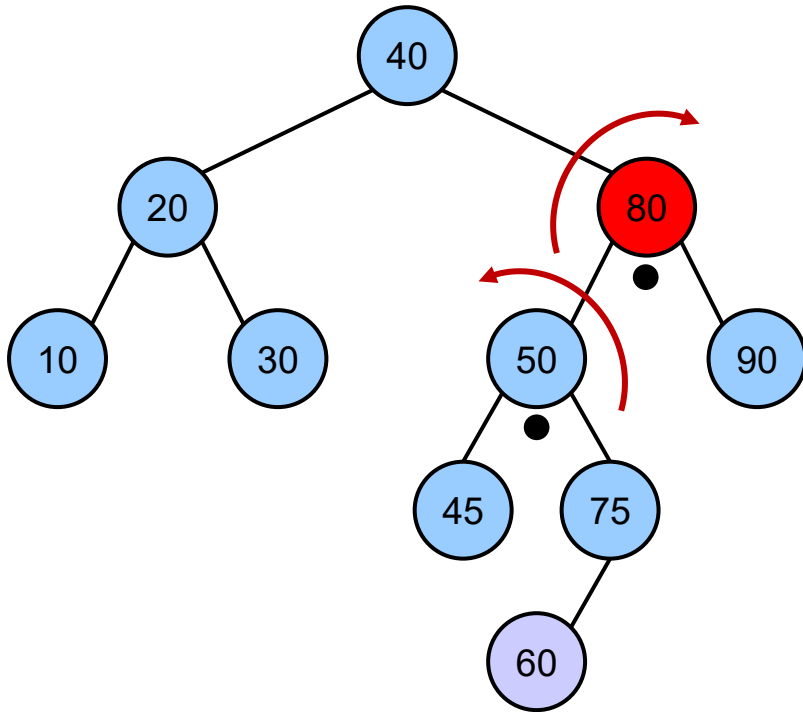
Inserción **Derecha - Izquierda**

Se arregla con rotación **Izquierda - Derecha**



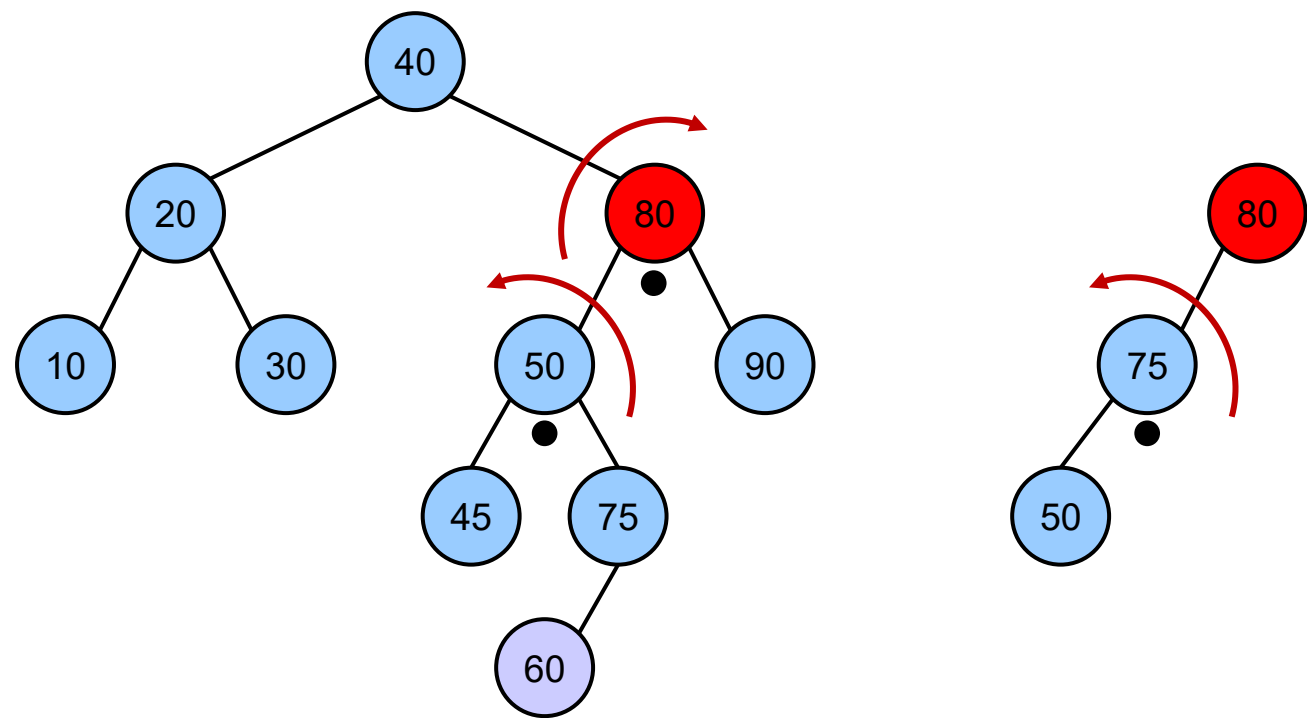
Inserción **Derecha - Izquierda**

Se arregla con rotación **Izquierda - Derecha**



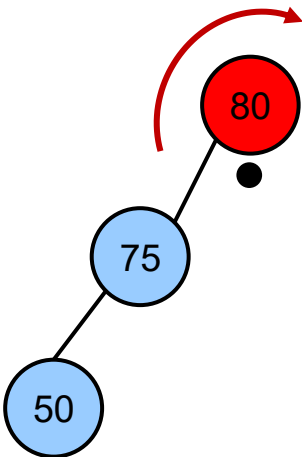
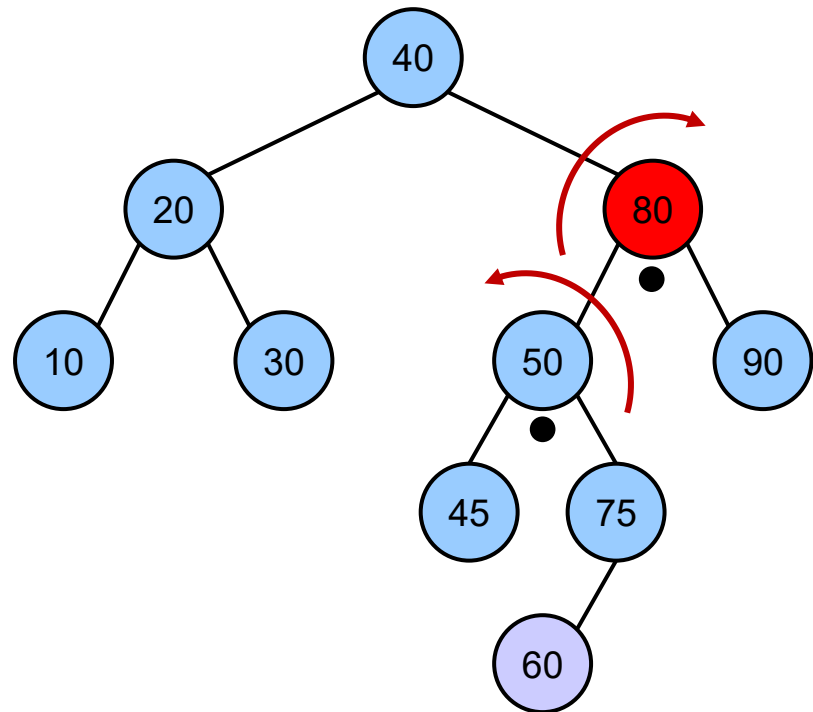
Inserción **Derecha - Izquierda**

Se arregla con rotación **Izquierda - Derecha**



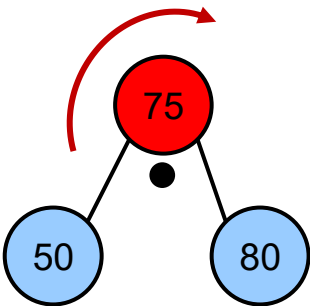
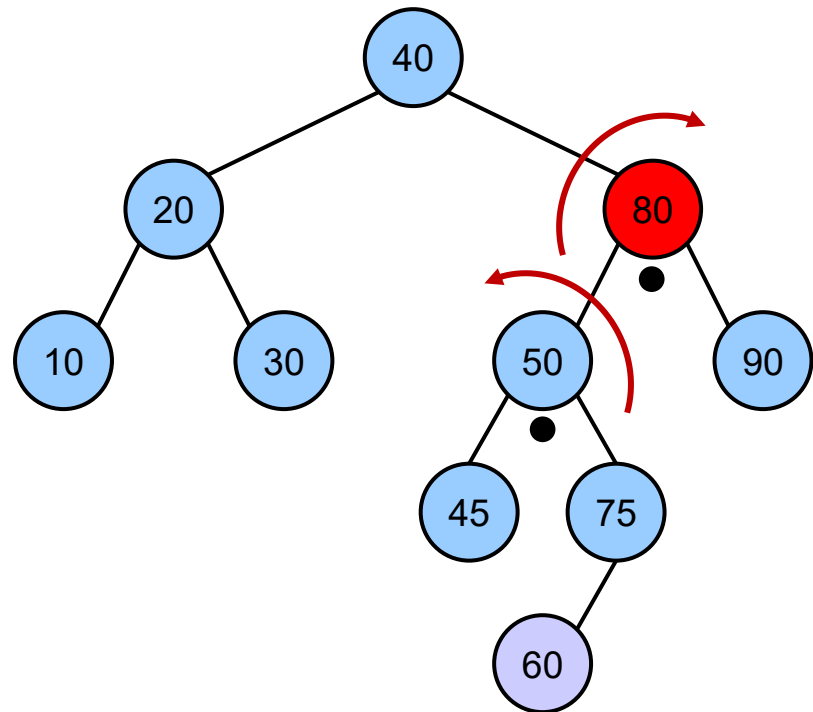
Inserción **Derecha - Izquierda**

Se arregla con rotación **Izquierda - Derecha**



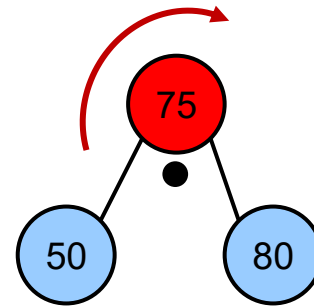
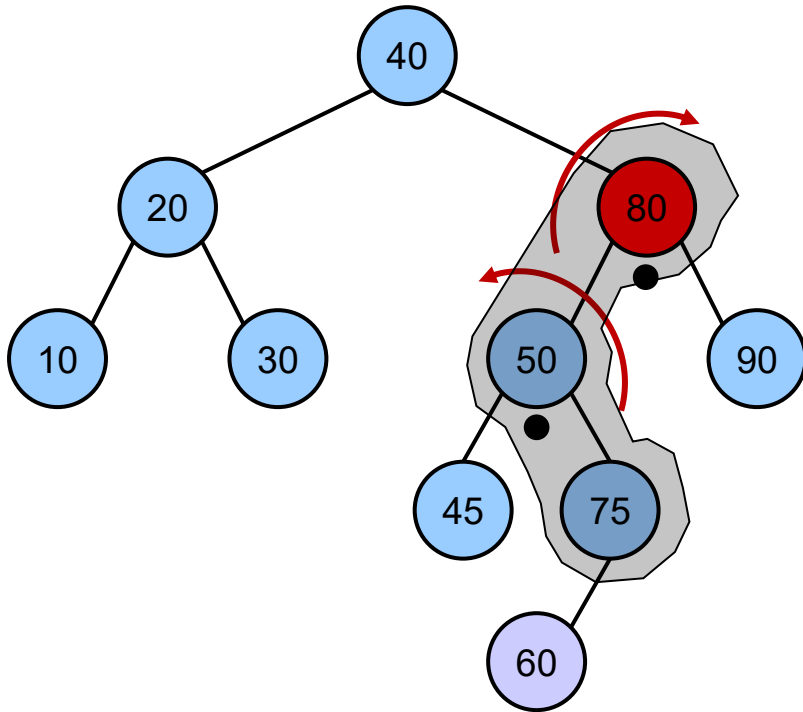
Inserción **Derecha - Izquierda**

Se arregla con rotación **Izquierda - Derecha**



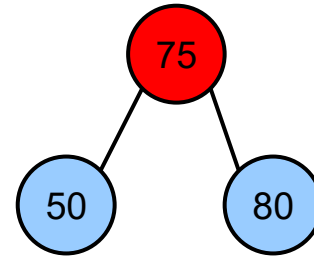
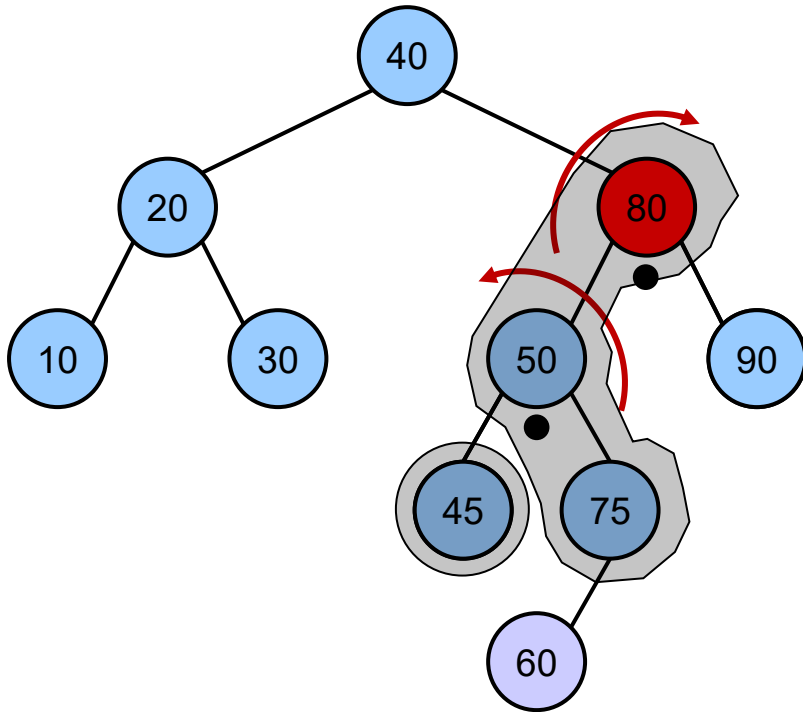
Inserción **Derecha - Izquierda**

Se arregla con rotación **Izquierda - Derecha**



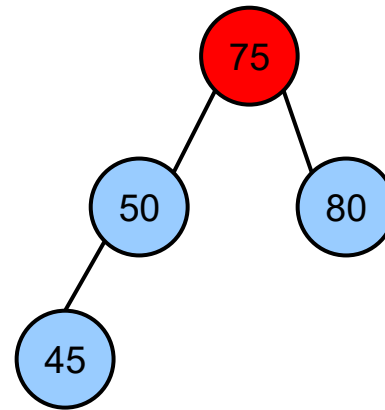
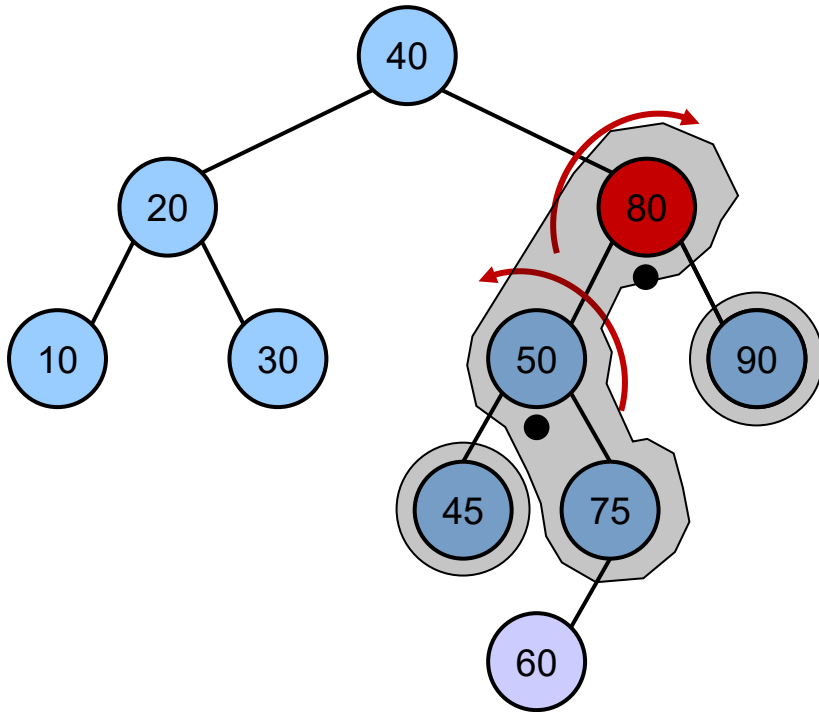
Inserción **Derecha** - **Izquierda**

Se arregla con rotación **Izquierda** - **Derecha**



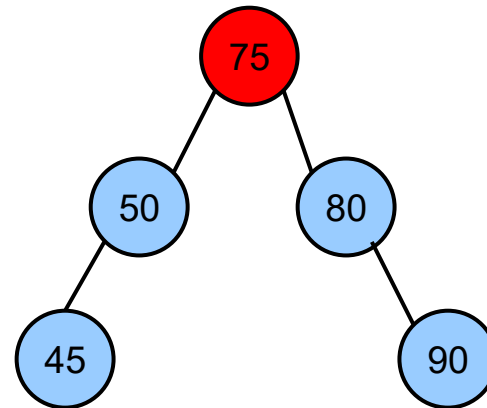
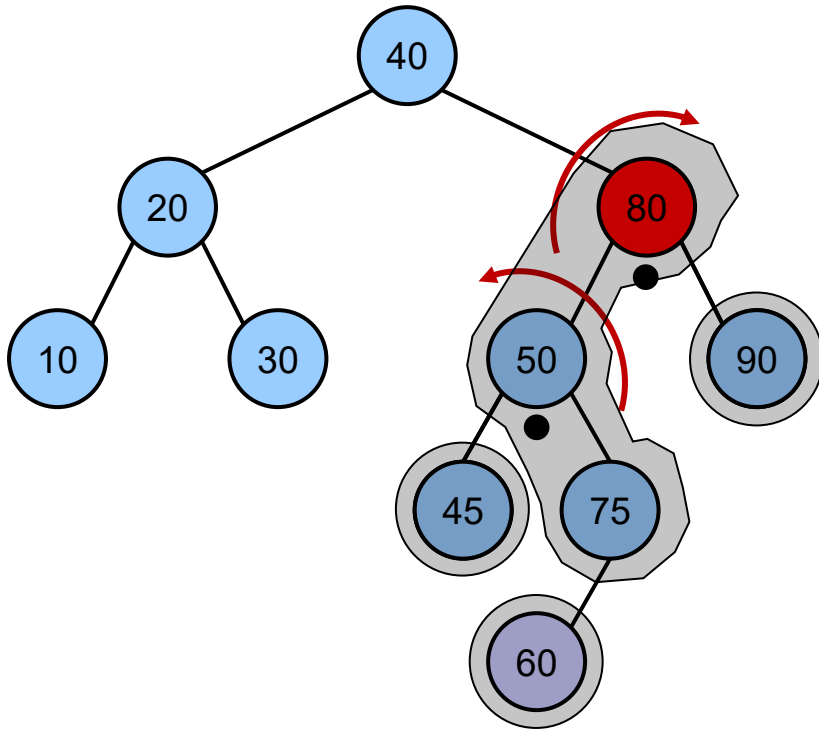
Inserción **Derecha** - **Izquierda**

Se arregla con rotación **Izquierda** - **Derecha**



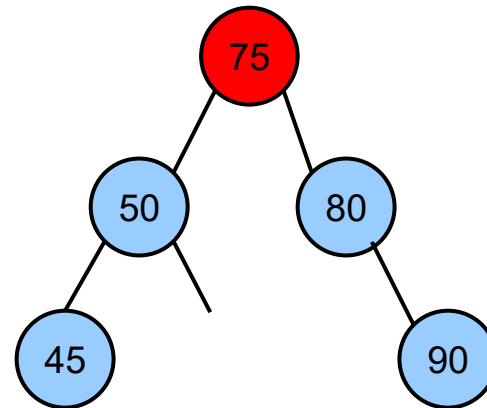
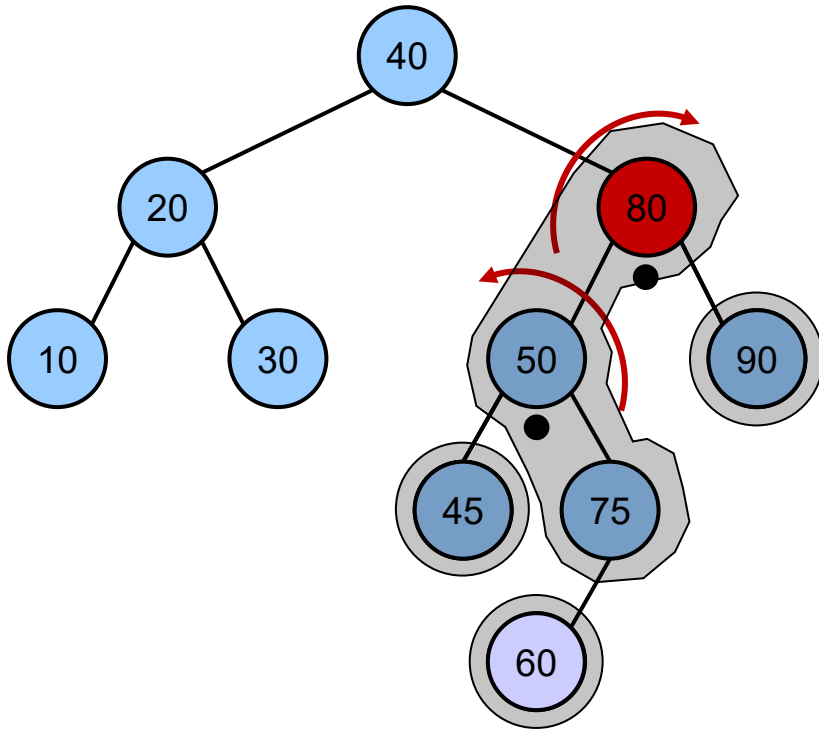
Inserción **Derecha** - **Izquierda**

Se arregla con rotación **Izquierda** - **Derecha**



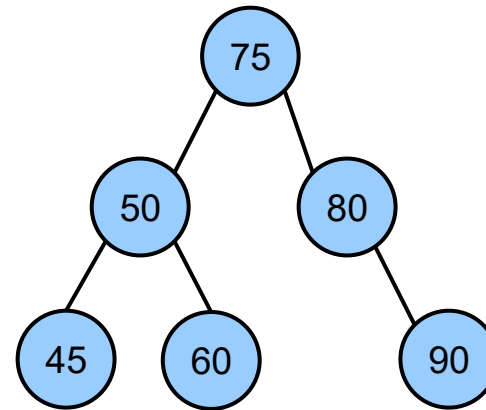
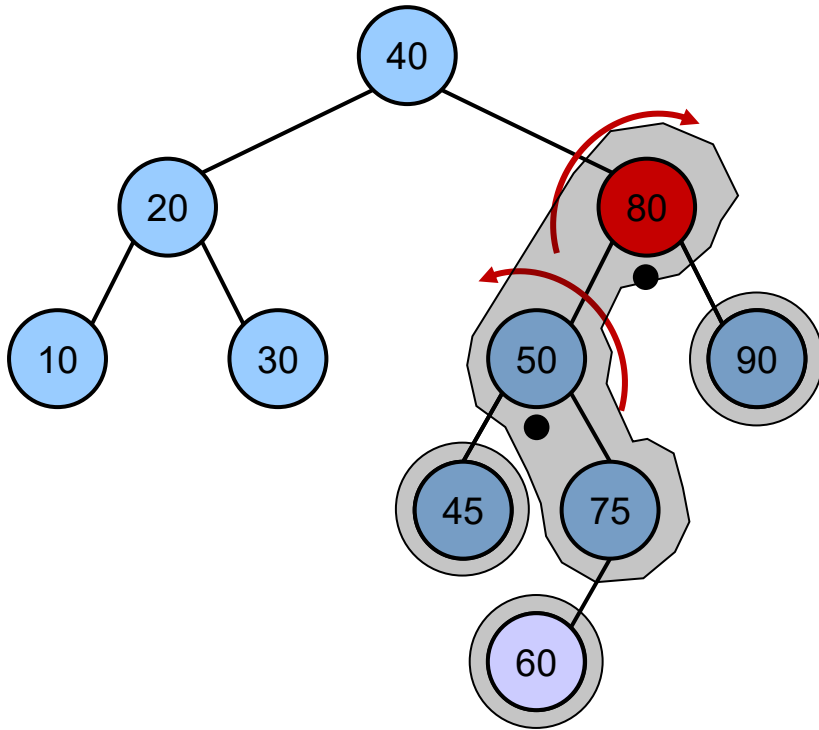
Inserción **Derecha - Izquierda**

Se arregla con rotación **Izquierda - Derecha**



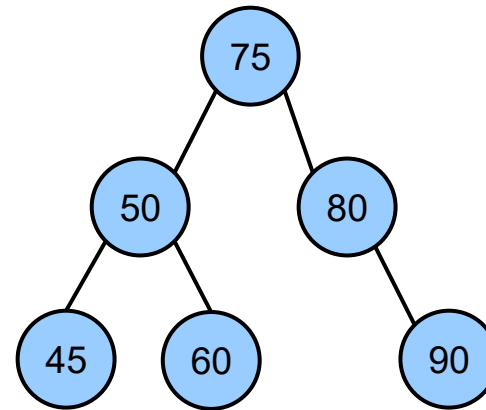
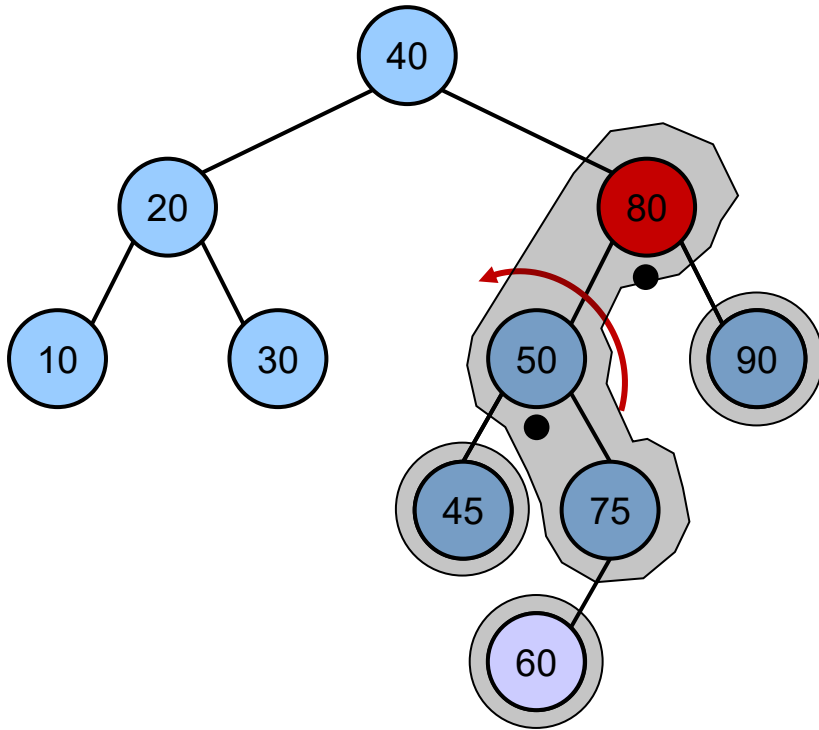
Inserción **Derecha** - **Izquierda**

Se arregla con rotación **Izquierda** - **Derecha**



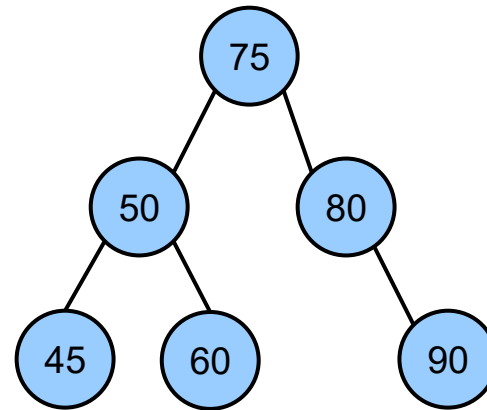
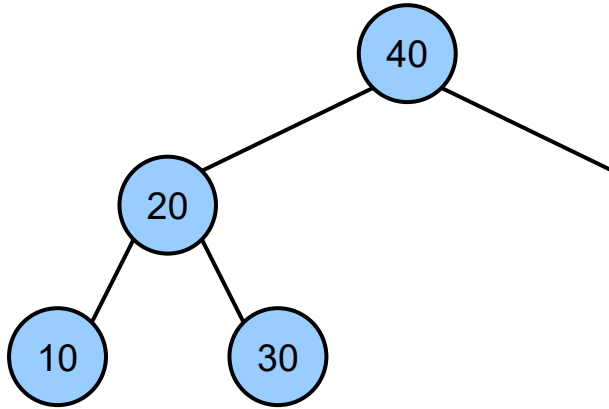
Inserción **Derecha - Izquierda**

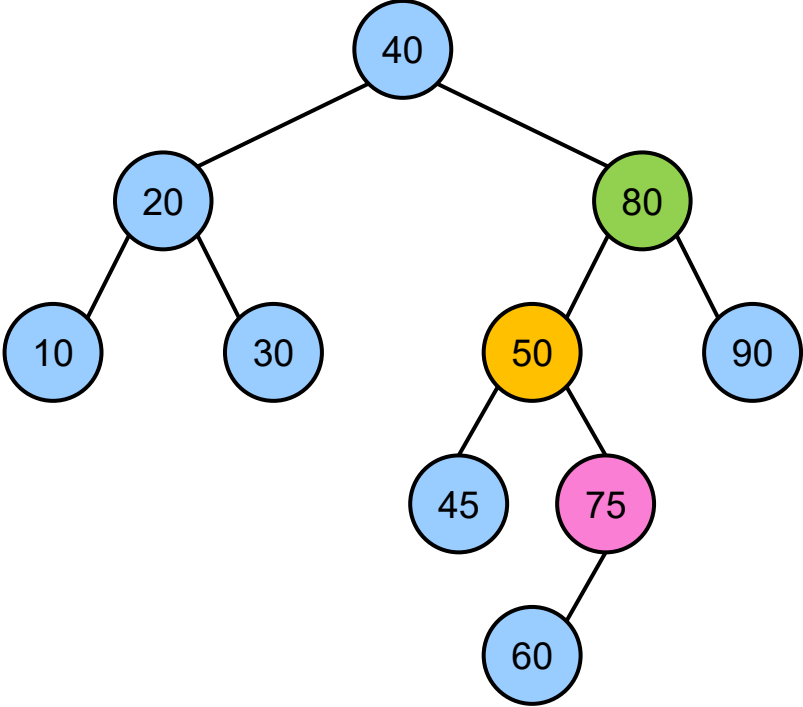
Se arregla con rotación **Izquierda - Derecha**

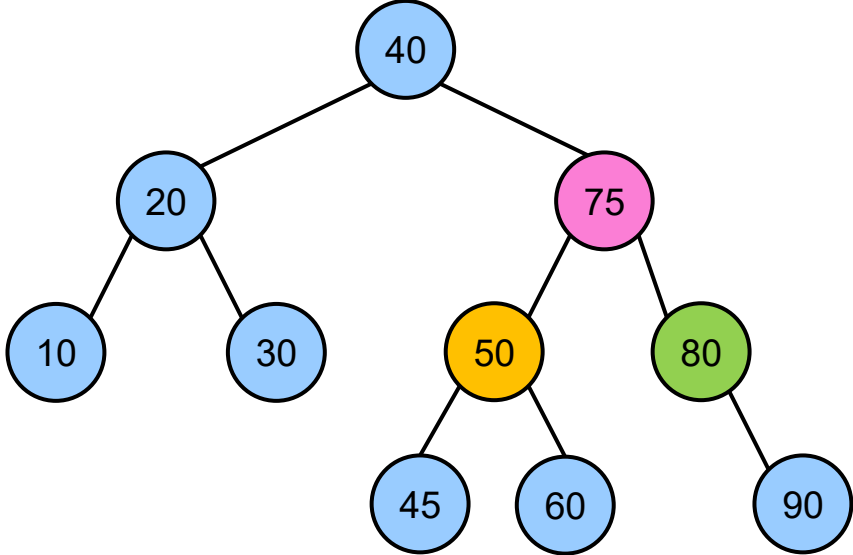
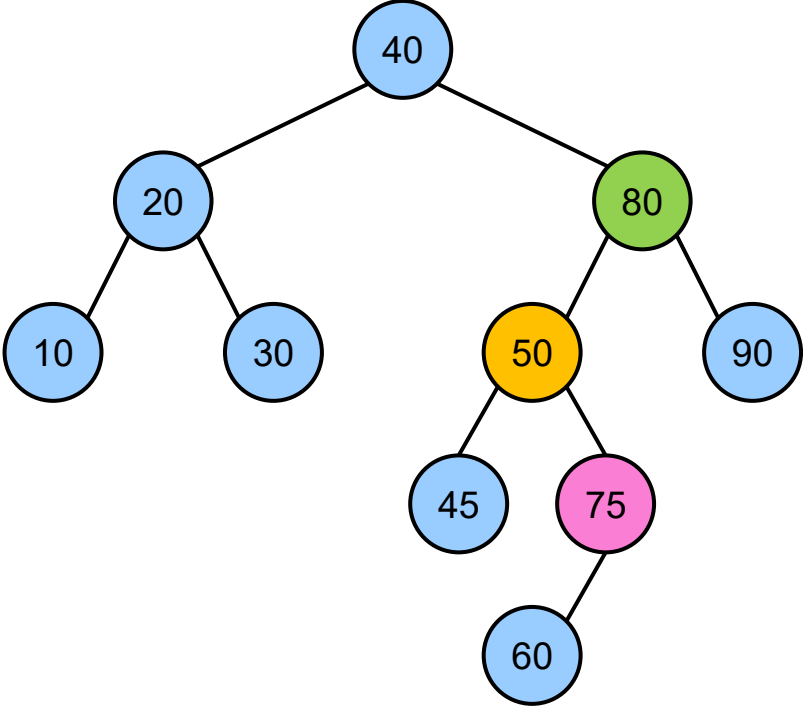


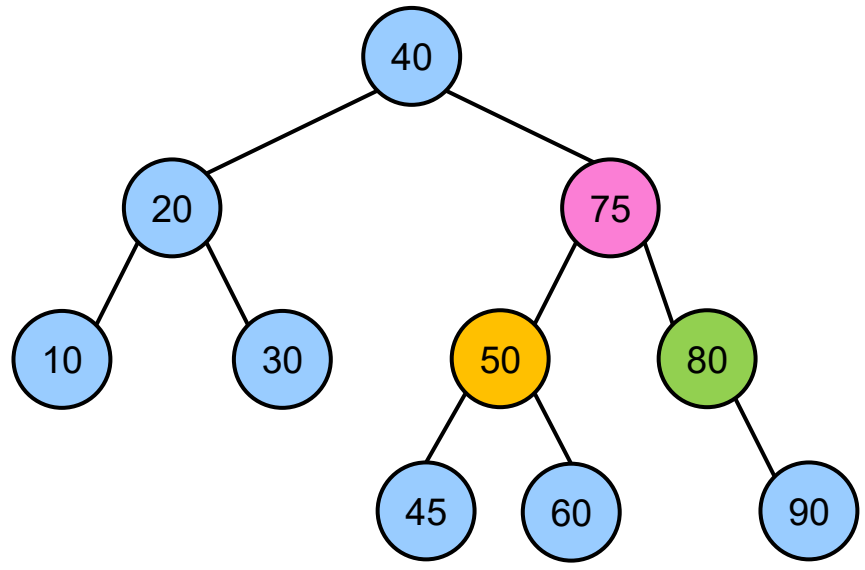
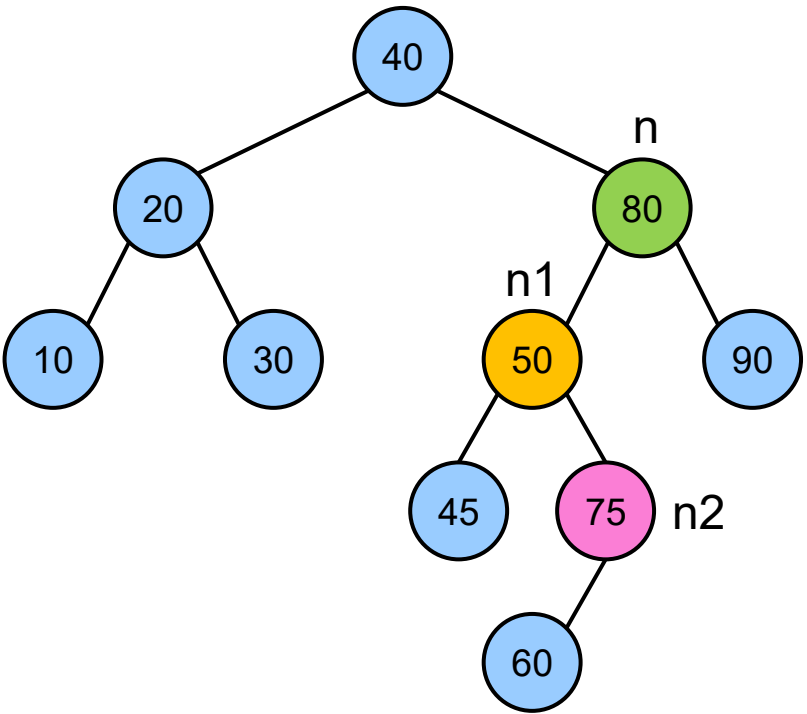
Inserción **Derecha - Izquierda**

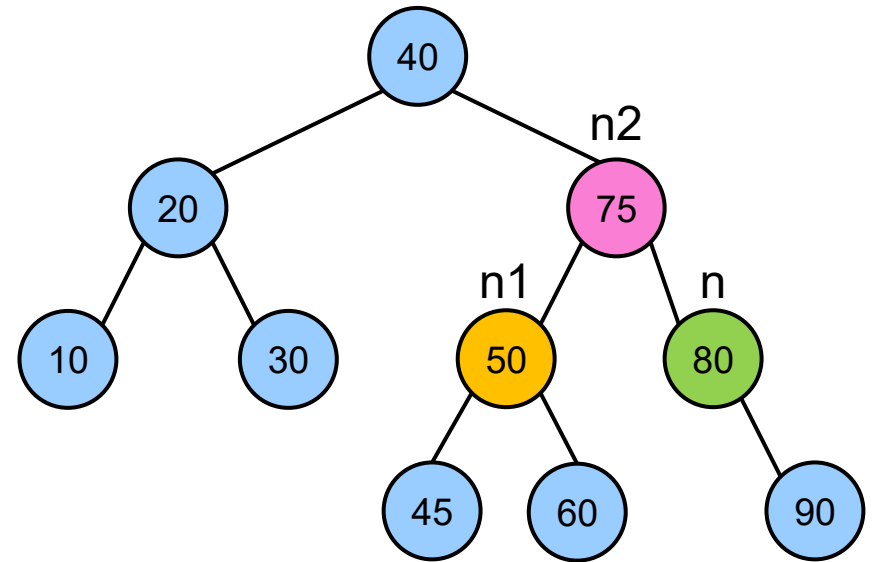
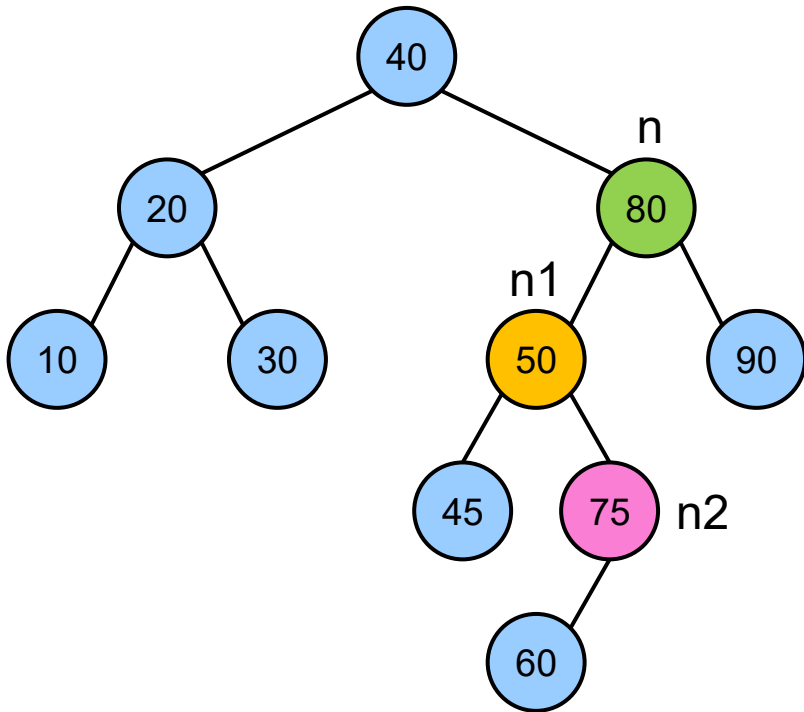
Se arregla con rotación **Izquierda - Derecha**











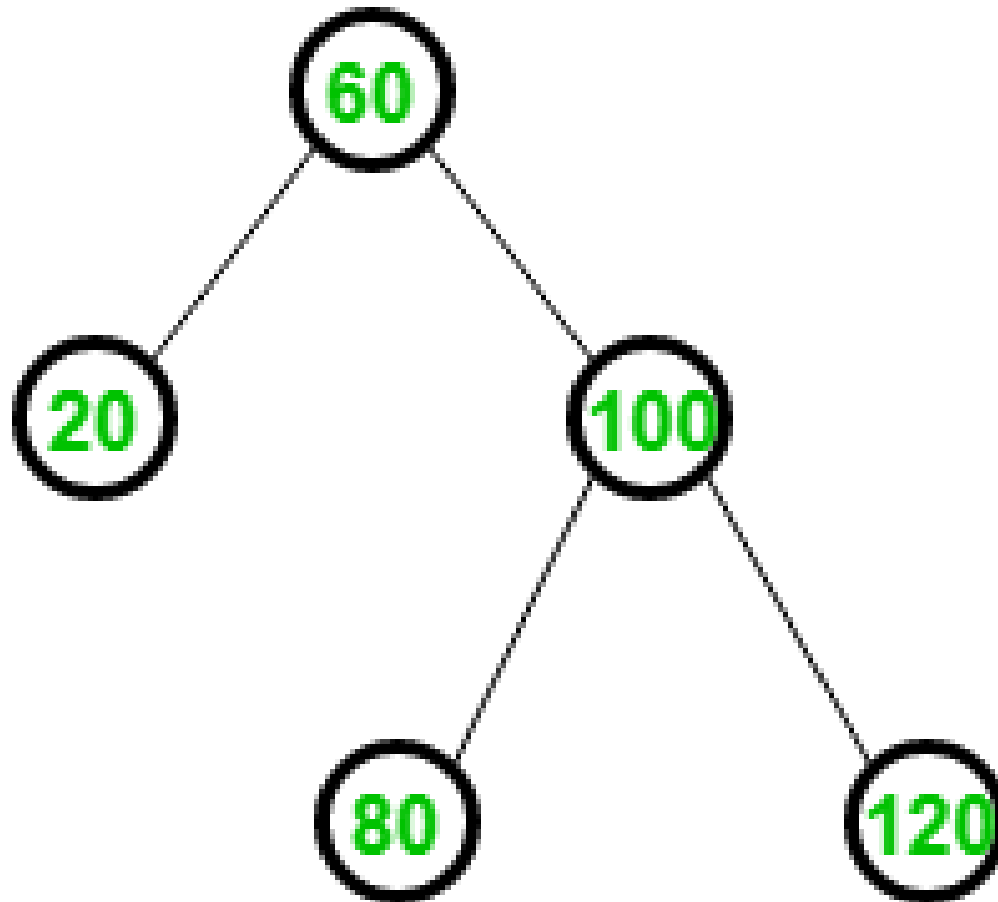
```

n1.derecho = n2.izquierdo
n2.izquierdo = n1
n.izquierdo = n2.derecho
n2.derecho = n
n = n2

```

Ejercicio

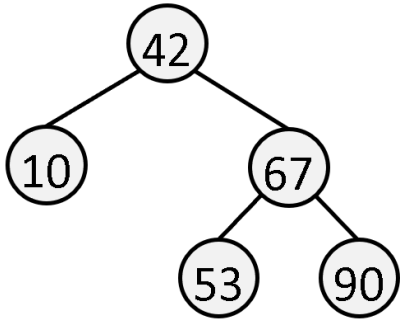
¿Cuál es el árbol resultante después de insertar la clave **70** en el árbol AVL mostrado a continuación?



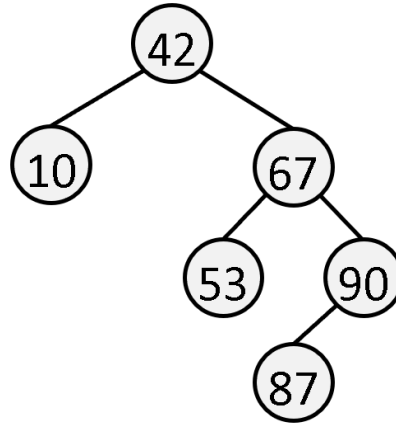
Ejercicio

¿Cuál de los siguientes árboles está(n) balanceado(s)?:

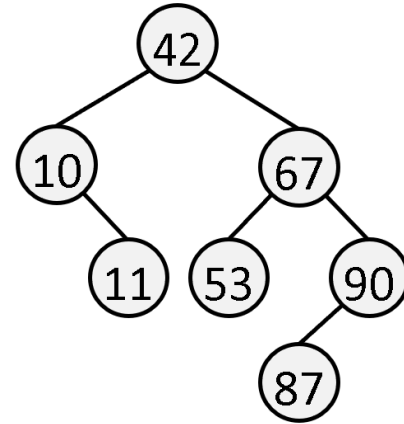
T1



T2



T3



Ejercicio

